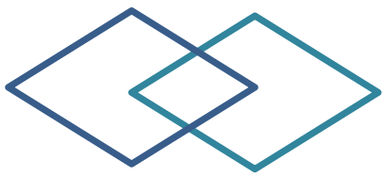


第9章

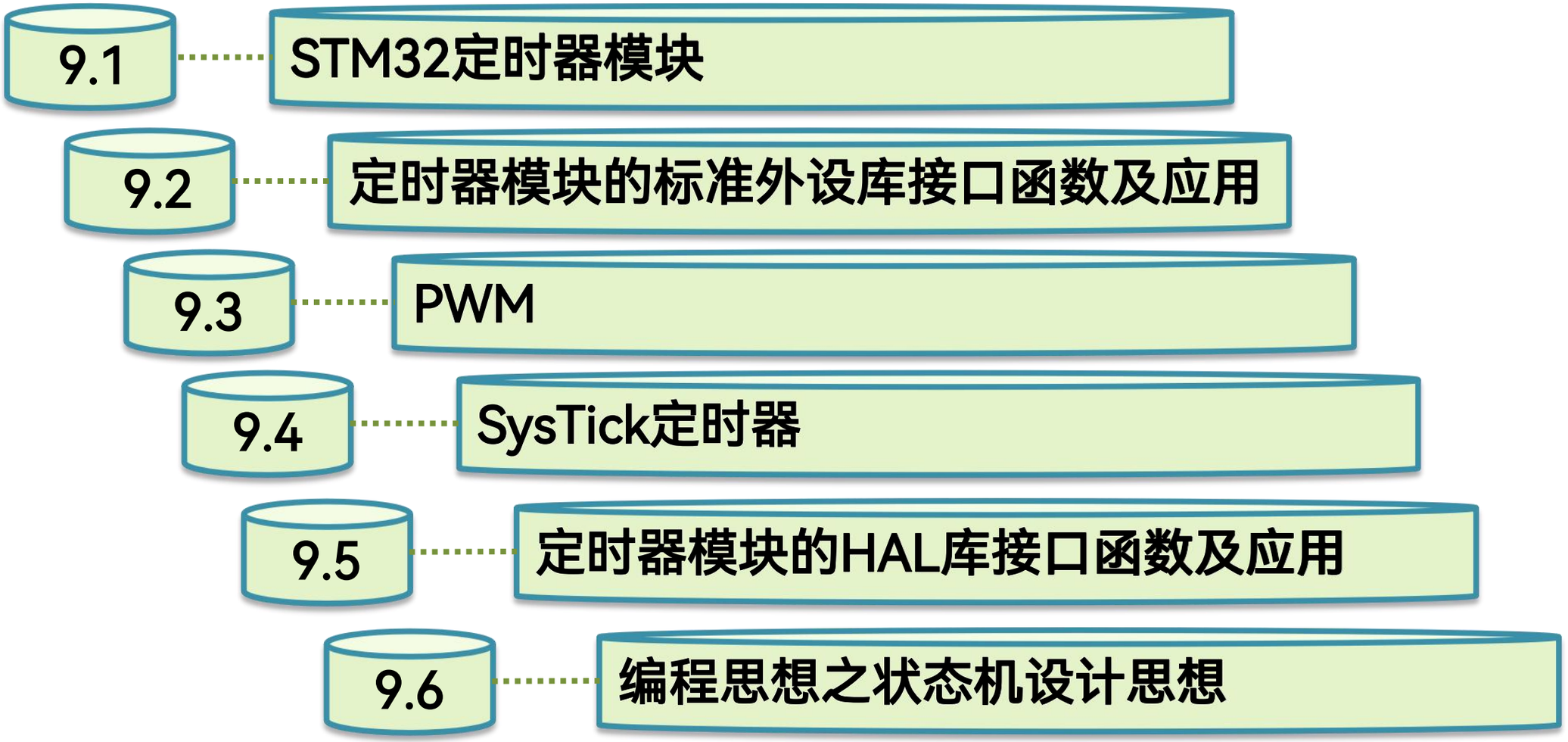
定 时 器



本章知识与能力要求

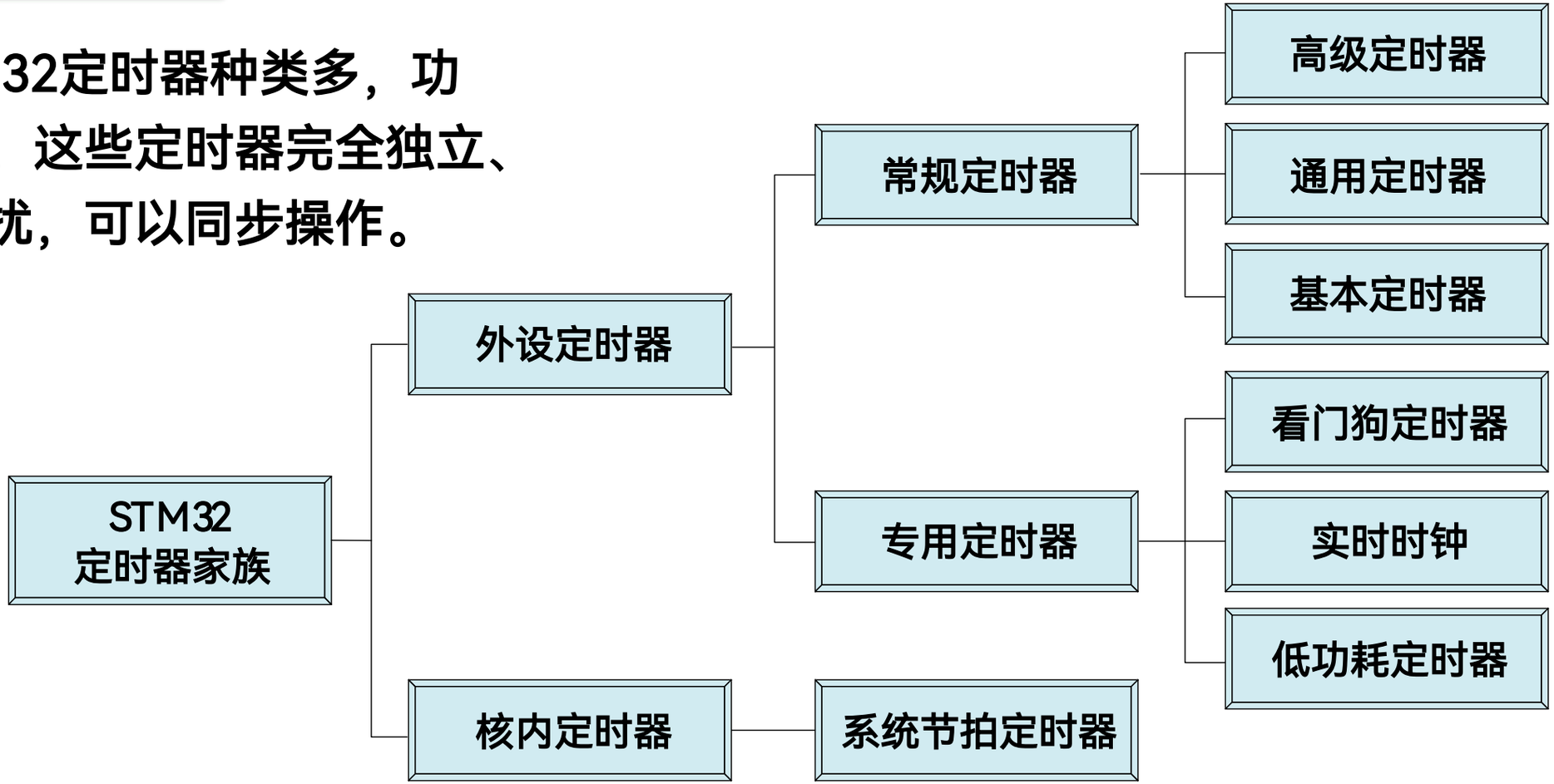
- 了解和掌握定时器的基本功能；
- 理解和掌握STM32定时器的内部结构、工作模式和主要特性；
- 掌握STM32通用定时器延时的工作原理；
- 熟悉STM32标准外设库和HAL库中有关定时器的库函数；
- 掌握SysTick定时器定时功能；
- 掌握基于HAL库实现定时器精确延时功能。

第9章 定时器


9.1
STM32定时器模块
9.2
定时器模块的标准外设库接口函数及应用
9.3
PWM
9.4
SysTick定时器
9.5
定时器模块的HAL库接口函数及应用
9.6
编程思想之状态机设计思想

第9章 定时器

STM32定时器种类多，功能强大，这些定时器完全独立、互不干扰，可以同步操作。



第9章 定时器

SysTick定时器

SysTick系统时钟位于Cortex-M3内核，是一个24位的递减计数器，主要用于：

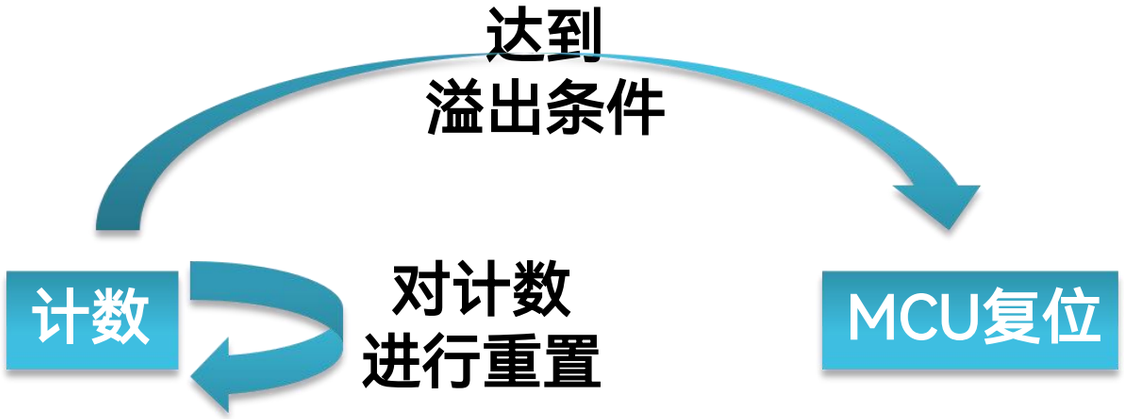
- 精确延时，在多任务操作系统中为系统提供时间基准(时基）；
- 任务切换，为每个任务分配时间片。

WatchDog看门狗

- 作用：当微控制器受到外部干扰或程序中出现不可预知的逻辑故障导致应用程序脱离正常的执行流程时（俗称程序跑飞），在一定的时间间隔内使系统复位，回到初始状态；
- 看门狗设计是用来监视MCU程序运行状态的，是确保系统可靠稳定运行的一种有效措施。

第9章 定时器

WatchDog看门狗



看门狗

- 独立看门狗 IWDG
  - 采用独立时钟，不受其他时钟和总线影响，可在停机和待机模式下工作
    - 适用于独立于主程序之外、能够独立工作且对时间精度要求不高的场合
- 窗口看门狗 WWDG
  - 计数值有一个上限时间和下限时间，低于下限时间或高于上限时间都会触发MCU复位
    - 适用于精确计时窗口起作用的应用程序

9.1 STM32定时器模块



高级定时器的功能强于通用定时器，通用定时器的功能强于基本定时器。

9.1 STM32定时器模块

STM32定时器分类比较表

定时器	基本定时器 (TIM6、TIM7)	通用定时器 TIMx(x=2~5)	高级定时器 (TIM1、TIM8)
计数器类型	16位，向上	16位， 向上、 向下、 向上/向下	16位， 向上、 向下、 向上/向下
预分频系数	1～65535 之间的任意数	1～65535 之间的任意数	1～65535 之间的任意数
输入/捕获通道	无	四个独立通道：输入捕获、输出比较、 PWM生成、单脉冲模式输出	
产生中断/DMA	可以	可以	可以
刹车(电机控制)	无	无	可以

9.1 STM32定时器模块

定时器的主要功能

计数	定时	输入捕获	输出比较
脉冲计数，使用使用微控制器内部的外部时钟（PCLK）来计数，是对固定周期的脉冲信号计数。	时间控制，通过对微控制器内部的时钟脉冲进行计数实现定时功能。	对输入信号进行捕获，实现对脉冲的频率测量，可用于对外部输入信号脉冲宽度的测量，比如测量电机转速。	将计数器计数值和设定值进行比较，根据比较结果输出不同电平，用于控制输出波形，比如直流电机的调速。

9.1 STM32定时器模块

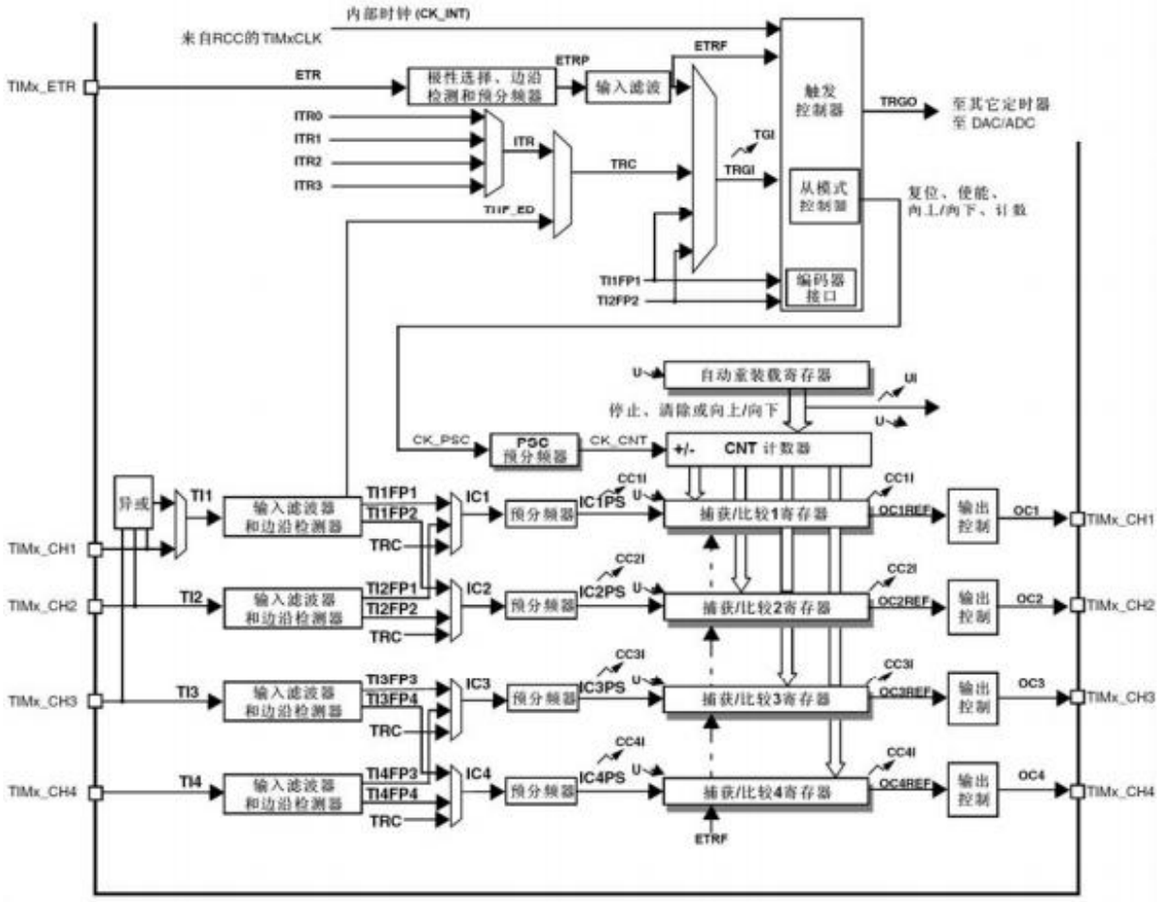
1 9.1.1 通用定时器

2 9.1.2 基本定时器

3 9.1.3 高级定时器

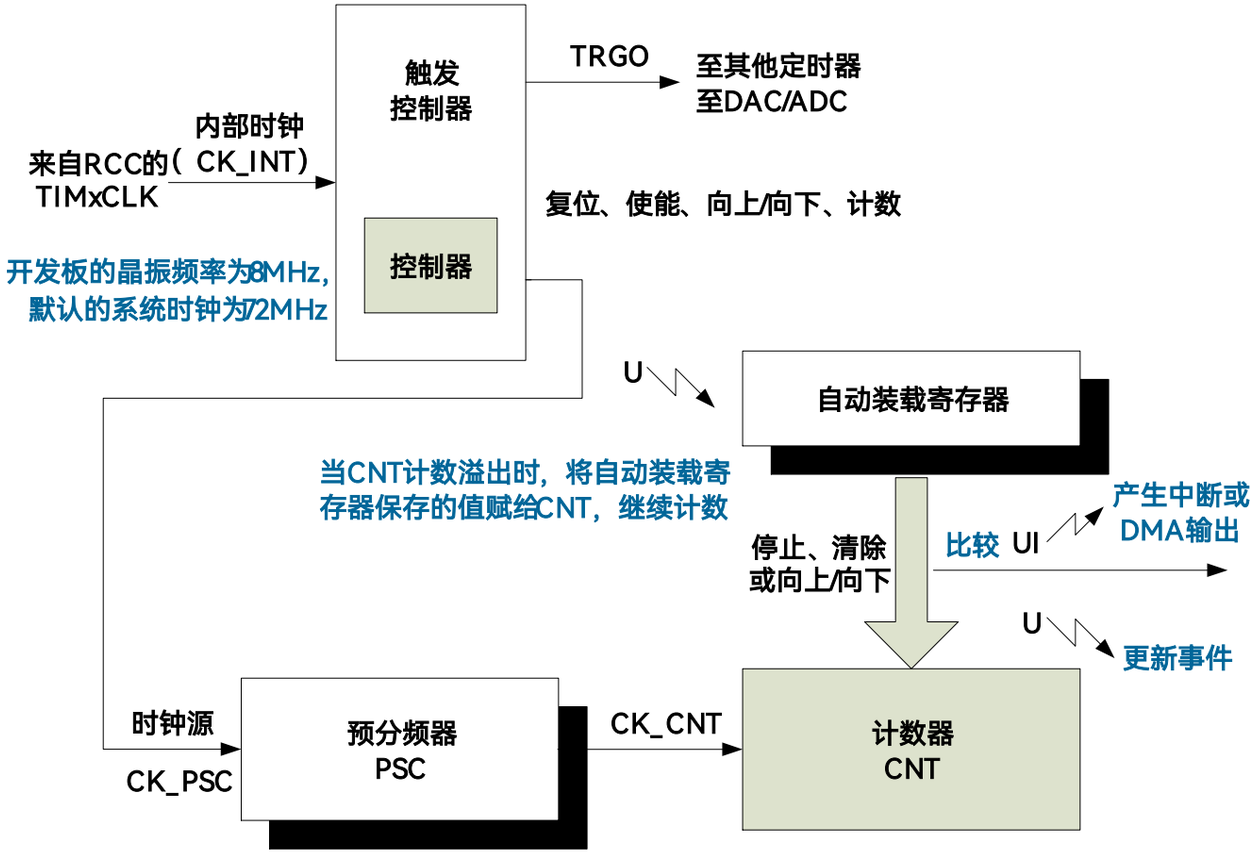
9.1.1 通用定时器

TIM2、TIM3、TIM4、TIM5为STM32的4个独立的16位**通用定时器**，具有定时、测量输入信号的脉冲长度(输入捕获）、输出所需波形(输出比较、产生PWM、单脉冲输出等）等功能。



通用定时器内部结构框图

9.1.1 通用定时器



通用定时器内部简化图

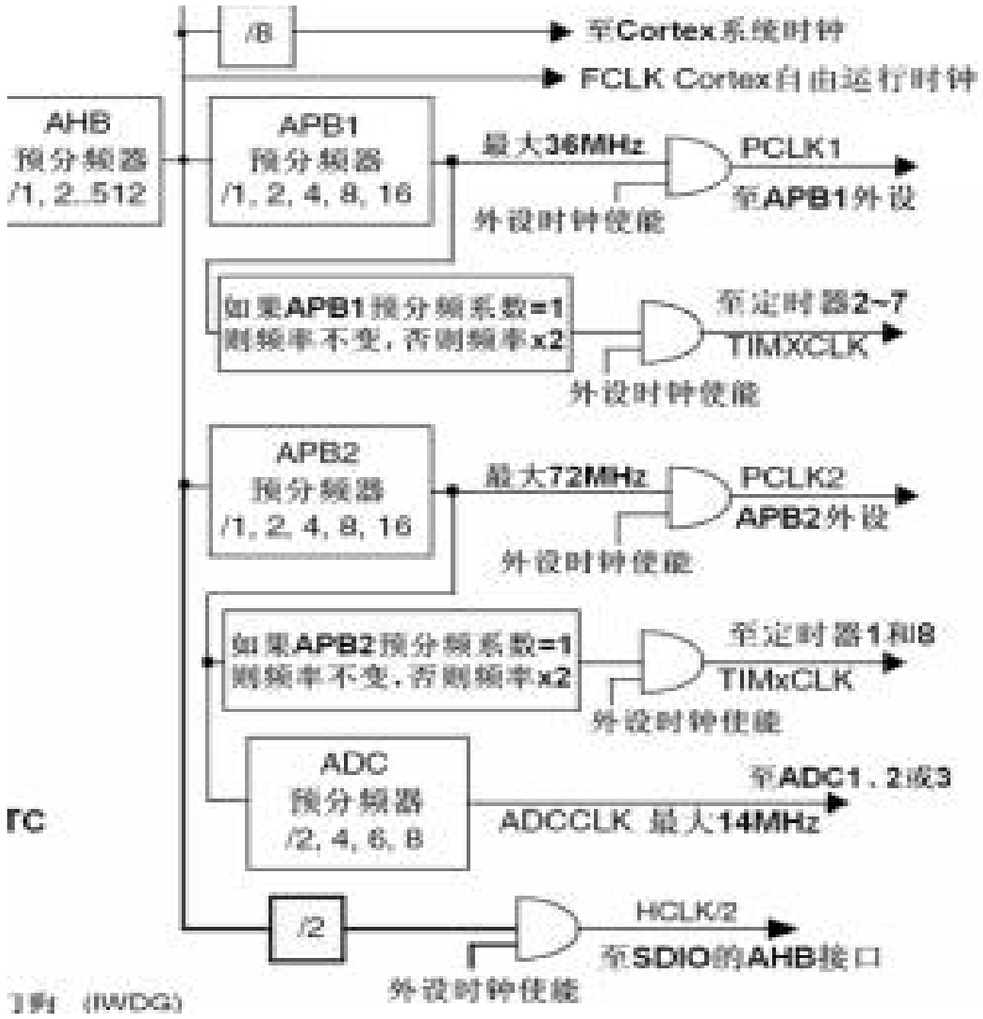
STM32F103系列微控制器的定时器功能十分强大，内部结构也比较复杂，STM32通用定时器TIMx(x=2，3，4，5)主要由时钟源、时钟单元、捕获和比较通道等构成，核心是可编程预分频驱动的**16位自动装载计数器**。

9.1.1 通用定时器

1 时钟源

当定时器使用内部时钟时，定时器的时钟源统称为TIMxCLK。虽然在系统默认的配置中，TIMxCLK的时钟频率都是72MHz，但其**时钟来源**并不相同。

- 定时器TIM2~TIM7挂接在APB1上
- 定时器TIM1和TIM8挂接在APB2上



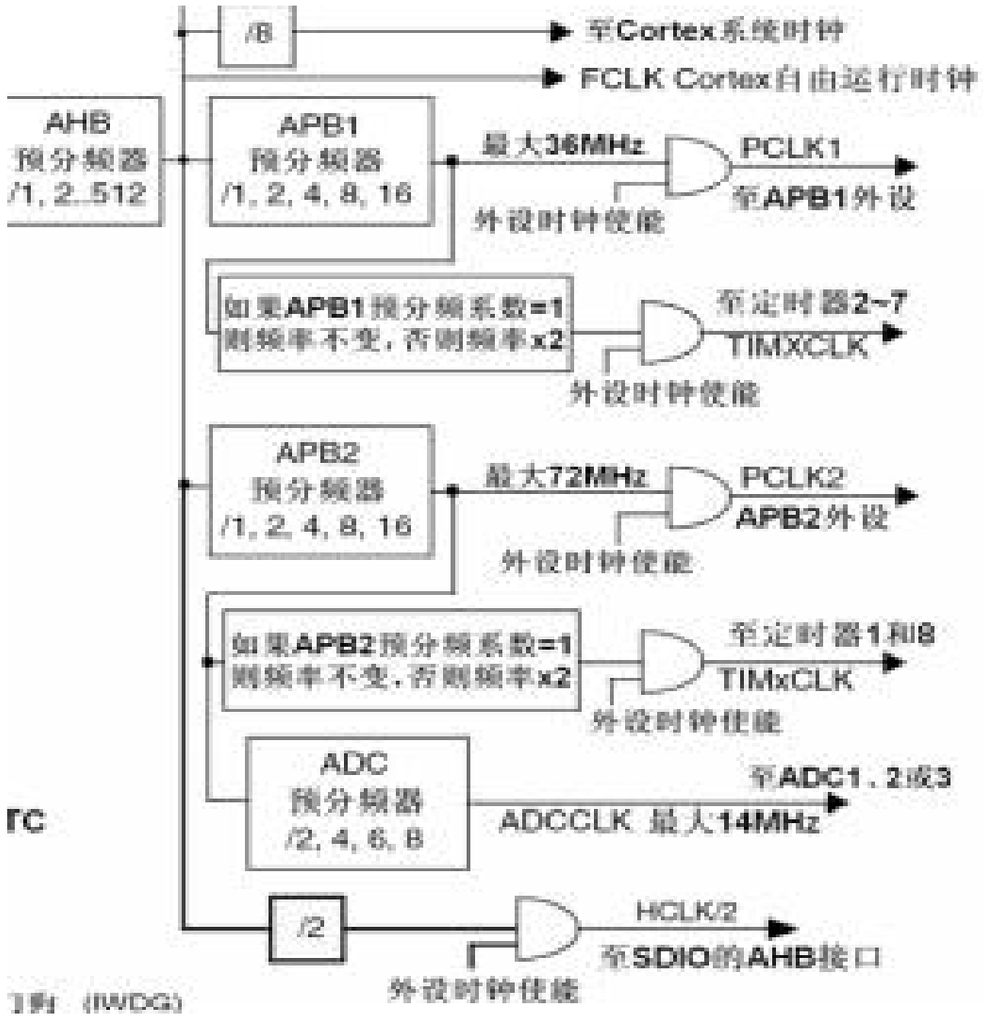
通用定时器内部结构框图

9.1.1 通用定时器

1 时钟源

若外部晶振的频率为8MHz，则系统默认的时钟频率为72MHz

- APB1预分频器的分频系数设置为2，则PCLK1=36MHz；
- APB2预分频系数设置为1，则PCLK2=72MHz，TIM1和TIM8的时钟频率TIMxCLK=72MHz；
- Cortex系统时钟由AHB时钟(HCLK)8分频得到，即SysTick的频率为9MHz。



通用定时器内部结构框图

9.1.1 通用定时器

2 预分频器PSC

可以以1 ~ 65535之间的任意数值对时钟源CK_PSC的时钟频率进行分频，输出CK_CNT脉冲供计数器CNT进行计数。

3 计数器CNT

- TIMxCNT是一个16位的寄存器，计数范围为1 ~ 65535，可以向上计数、向下计数或向下向上双向计数。
- 要得到想要的计数值，需要对输入时钟频率进行分频。
- 当计数值达到设定值时，便产生溢出事件，溢出时产生中断或DMA请求，然后再由自动装载寄存器进行重新加载或更新。
- 计数器溢出中断属于软件中断，执行相应的定时器中断服务程序。

9.1.1 通用定时器

4 自动装载寄存器ARR

定时器的定时时间主要取决于定时周期和预分频因子，**计算公式**为：

定时时间=(ARR+1）×(预分频值PSC+1）/输入时钟频率

或　　T=(TIM_Period +1）*(TIM_Prescaler +1）/TIMxCLK

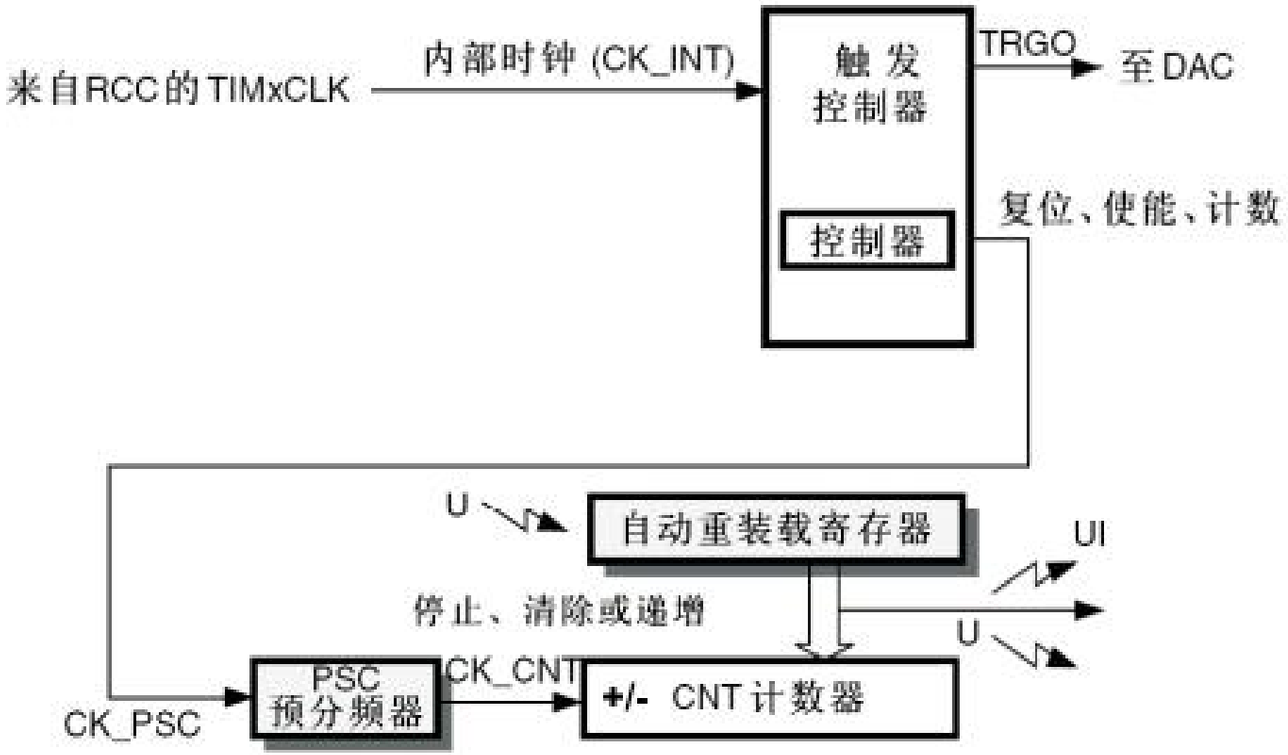
这里ARR+1是因为计数器都是从0开始计数的。

例如：使用通用定时器定时1s。假设系统时钟为72MHz，通用定时器时钟TIMxCLK为72MHz，设置如下：

预分频系数PSC=6000-1； ARR=2000-1；

则，定时时间=2000×36000/72000000=1s。

9.1.2 基本定时器



STM32有2个基本定时器TIM6和TIM7，可用作：

- 通用的16位计数器：
- 产生DAC触发信号

基本定时器的计数模式只有向上计数模式。

9.1.3 高级定时器

TIM1和TIM8是STM32的2个16位的高级定时器

高级定时器相比基本定时器、通用定时器，功能更为强大。

作用：通用定时功能

TIM1还提供控制三相六步电机的接口，具有刹车功能、死区时间控制等功能，主要用于电机控制。

9.2 定时器模块的标准外设库接口函数及应用

9.2.1 定时器标准外设库接口函数

9.2.2 定时器标准外设库配置过程

9.3.3 定时器标准外设库应用实例

9.2.1 定时器标准外设库接口函数

STM32F103系列微控制器的定时器库函数存放在STM32标准外设库的stm32f10x_tim.c和stm32f10x_tim.h文件中，stm32f10x_tim.h头文件中声明了定时器有关的库函数以及相关的宏定义、结构体等。

由于STM32的定时器功能较为强大，其库函数也较多。

```
1054 void TIM_DeInit(TIM_TypeDef* TIMx);
1055 void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct);
1056 void TIM_OC1Init(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* TIM_OCInitStruct);
1057 void TIM_OC2Init(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* TIM_OCInitStruct);
1058 void TIM_OC3Init(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* TIM_OCInitStruct);
1059 void TIM_OC4Init(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* TIM_OCInitStruct);
1060 void TIM_ICInit(TIM_TypeDef* TIMx, TIM_ICInitTypeDef* TIM_ICInitStruct);
1061 void TIM_PWMIConfig(TIM_TypeDef* TIMx, TIM_ICInitTypeDef* TIM_ICInitStruct);
1062 void TIM_BDTRConfig(TIM_TypeDef* TIMx, TIM_BDTRInitTypeDef *TIM_BDTRInitStruct);
1063 void TIM_TimeBaseStructInit(TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct);
1064 void TIM_OCStructInit(TIM_OCInitTypeDef* TIM_OCInitStruct);
1065 void TIM_ICStructInit(TIM_ICInitTypeDef* TIM_ICInitStruct);
1066 void TIM_BDTRStructInit(TIM_BDTRInitTypeDef* TIM_BDTRInitStruct);
1067 void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState NewState);
1068 void TIM_CtrlPWMOutputs(TIM_TypeDef* TIMx, FunctionalState NewState);
1069 void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState NewState);
1070 void TIM_GenerateEvent(TIM_TypeDef* TIMx, uint16_t TIM_EventSource);
1071 void TIM_DMAConfig(TIM_TypeDef* TIMx, uint16_t TIM_DMABase, uint16_t TIM_DMABurstLength);
1072 void TIM_DMACmd(TIM_TypeDef* TIMx, uint16_t TIM_DMASource, FunctionalState NewState);
1073 void TIM_InternalClockConfig(TIM_TypeDef* TIMx);
1074 void TIM_ITRxExternalClockConfig(TIM_TypeDef* TIMx, uint16_t TIM_InputTriggerSource);
1075 void TIM_TIxExternalClockConfig(TIM_TypeDef* TIMx, uint16_t TIM_TIxExternalCLKSource,
1076                                 uint16_t TIM_ICPolarity, uint16_t ICFilter);
1077 void TIM_ETRClockMode1Config(TIM_TypeDef* TIMx, uint16_t TIM_ExtTRGPrescaler, uint16_t TIM_ExtTRGPolarity,
1078                              uint16_t ExtTRGFilter);
1079 void TIM_ETRClockMode2Config(TIM_TypeDef* TIMx, uint16_t TIM_ExtTRGPrescaler,
1080                              uint16_t TIM_ExtTRGPolarity, uint16_t ExtTRGFilter);
1081 void TIM_ETRConfig(TIM_TypeDef* TIMx, uint16_t TIM_ExtTRGPrescaler, uint16_t TIM_ExtTRGPolarity,
1082                    uint16_t ExtTRGFilter);
1083 void TIM_PrescalerConfig(TIM_TypeDef* TIMx, uint16_t Prescaler, uint16_t TIM_PSCReloadMode);
1084 void TIM_CounterModeConfig(TIM_TypeDef* TIMx, uint16_t TIM_CounterMode);
1085 void TIM_SelectInputTrigger(TIM_TypeDef* TIMx, uint16_t TIM_InputTriggerSource);
1086 void TIM_EncoderInterfaceConfig(TIM_TypeDef* TIMx, uint16_t TIM_EncoderMode,
1087                                 uint16_t TIM_IC1Polarity, uint16_t TIM_IC2Polarity);
1088 void TIM_ForcedOC1Config(TIM_TypeDef* TIMx, uint16_t TIM_ForcedAction);
1089 void TIM_ForcedOC2Config(TIM_TypeDef* TIMx, uint16_t TIM_ForcedAction);
1090 void TIM_ForcedOC3Config(TIM_TypeDef* TIMx, uint16_t TIM_ForcedAction);
1091 void TIM_ForcedOC4Config(TIM_TypeDef* TIMx, uint16_t TIM_ForcedAction);
1092 void TIM_ARRPreloadConfig(TIM_TypeDef* TIMx, FunctionalState NewState);
1093 void TIM_SelectCOM(TIM_TypeDef* TIMx, FunctionalState NewState);
1094 void TIM_SelectCCDMA(TIM_TypeDef* TIMx, FunctionalState NewState);
1095 void TIM_CCPreloadControl(TIM_TypeDef* TIMx, FunctionalState NewState);
1096 void TIM_OC1PreloadConfig(TIM_TypeDef* TIMx, uint16_t TIM_OCPreload);
1097 void TIM_OC2PreloadConfig(TIM_TypeDef* TIMx, uint16_t TIM_OCPreload);
1098 void TIM_OC3PreloadConfig(TIM_TypeDef* TIMx, uint16_t TIM_OCPreload);
1099 void TIM_OC4PreloadConfig(TIM_TypeDef* TIMx, uint16_t TIM_OCPreload);
1100 void TIM_OC1FastConfig(TIM_TypeDef* TIMx, uint16_t TIM_OCFast);
```

9.2.1 定时器标准外设库接口函数

TIM_TimeBaseInit()库函数

用于定时器TIMx的参数初始化，函数原型：

```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef*
TIM_TimeBaseInitStruct);
        typedef struct
        {
          uint16_t TIM_Prescaler;
          uint16_t TIM_CounterMode;
          uint16_t TIM_Period;
          uint16_t TIM_ClockDivision;
          uint8_t
        TIM_RepetitionCounter;
        } TIM_TimeBaseInitTypeDef;
```

定义在stm32f10x_tim.h头文件中

TIMx预分频寄存器TIMx_PSC的值，数值上等于TIMx计数器TIMxCNT的预分频系数减1

为TIM1和TIM8高级定时器所特有

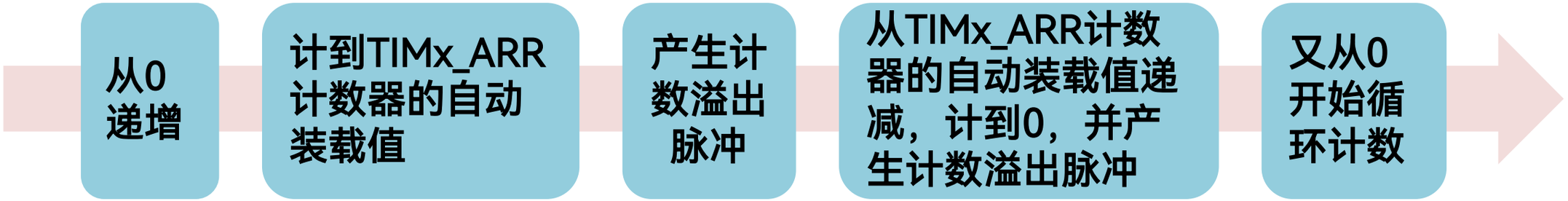
9.2.1 定时器标准外设库接口函数

TIM_CounterMode：TIMx计数器TIMxCNT的计数模式

TIM_CounterMode_Up	向上计数模式，从0递增，计到TIMx_ARR计数器的自动装载值，并产生计数溢出脉冲
TIM_CounterMode_Down	向下计数模式，从TIMx_ARR计数器的自动装载值递减，计到0，并产生计数溢出脉冲
TIM_CounterMode_CenterAligned1	中央对齐模式1计数模式
TIM_CounterMode_CenterAligned2	中央对齐模式2计数模式
TIM_CounterMode_CenterAligned3	中央对齐模式3计数模式

9.2.1 定时器标准外设库接口函数

3种中央对齐计数模式



- 中央对齐模式1计数器交替地向上和向下计数，输出比较中断标志位，只在计数器向下计数时被设置；
- 中央对齐模式2计数器交替地向上和向下计数，输出比较中断标志位，只在计数器向上计数时被设置；
- 中央对齐模式3计数器交替地向上和向下计数，输出比较中断标志位，在计数器向下和向上计数时均可被设置。

9.2.1 定时器标准外设库接口函数

下一个更新事件时装入自动重装载寄存器TIMx_ARR的周期值，数值上等于TIMx计数器TIMxCNT的计数周期减1。

TIM_Period是一个16位无符号整型数据，取值范围为0~65535。

定时器定时时间主要取决于预分频因子TIM_Prescaler和定时周期TIM_Period

T=(TIM_Prescaler+1）*(TIM_Period+1)

/TIMxCLK

TIMxCLK为定时器TIMx的时钟频率，STM32F103的TIMxCLK默认设置为72MHz。

```
typedef struct
{
  uint16_t TIM_Prescaler;
  uint16_t
TIM_CounterMode;
  uint16_t TIM_Period;
  uint16_t TIM_ClockDivision;
  uint8_t
TIM_RepetitionCounter;
} TIM_TimeBaseInitTypeDef;
```

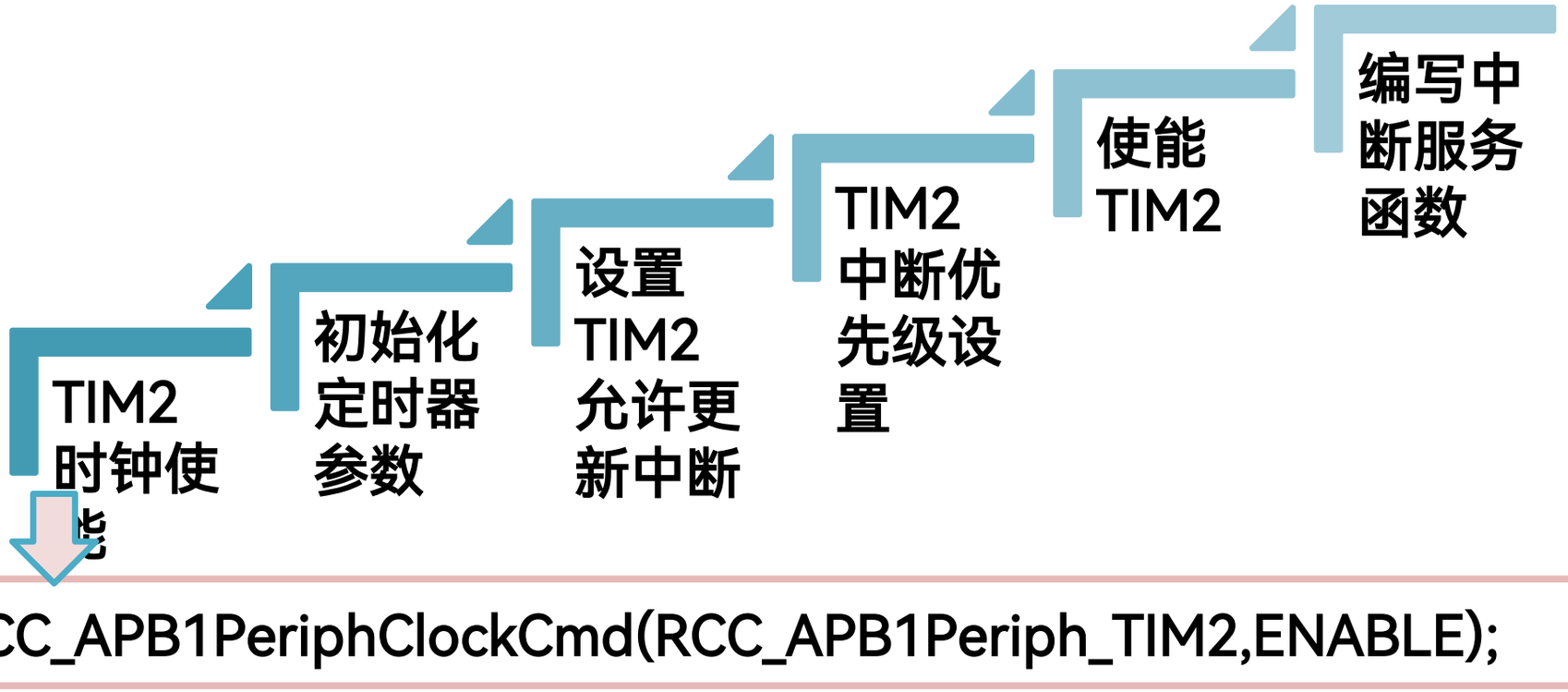
9.2.1 定时器标准外设库接口函数

与TIMx_CR寄存器中bit[9:8]的CKD[1:0]对应，用于计数器工作模式时滤除高频干扰，有如下选择：

- “00”——采样频率基准fDTS=定时器输入频率fCK_INT；
- “01”——fDTS=fCK_INT/2；
- “10”——fDTS=fCK_INT/4。

```
typedef struct
{
  uint16_t TIM_Prescaler;
  uint16_t TIM_CounterMode;
  uint16_t TIM_Period;
  uint16_t TIM_ClockDivision;
  uint8_t
TIM_RepetitionCounter;
} TIM_TimeBaseInitTypeDef;
```

9.2.2 定时器标准外设库配置过程



9.2.2 定时器标准外设库配置过程

Setp 2

初始化定时器参数，设置自动重装值，分频系数，计数方式

定时器初始化使用TIM_TimeBaseInit()函数

```
void TIM_TimeBaseInit(TIM_TypeDef*TIMx, TIM_TimeBaseInitTypeDef*
TIM_TimeBaseInitStruct);
```

第一个参数TIMx是具体定时器，x可为2~4

第二个参数为TimeBaseInitStruct结构体指针。

以初始化通用定时器TIM2为例，配置如下：

```
TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure; //定义TIM初始化类型结构体变量
TIM_TimeBaseStructure.TIM_Period = (36000-1); //设置自动重载计数周期值
TIM_TimeBaseStructure.TIM_Prescaler = (2000-1); //设置分频系数
TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1; //设置时钟分频因子
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //设置为向上计数方式
TIM_TimeBaseInit(TIM2,&TIM_TimeBaseStructure); //初始化定时器TIM2
TIM_Cmd(TIM2,ENABLE); //使能TIM2定时器
```

9.2.2 定时器标准外设库配置过程

Setp 3

设置TIM2允许更新中断

为了避免在初始化定时器时进入中断，需要在初始化过程中清除中断标志。

中断在使用前必须先使能再使用，如使能定时器TIM2的更新模式中断，则调用库函数TIM_ITConfig()，函数原型为：

```
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState NewState);
```

第一个参数TIMx是具体定时器，x可为2~4

第二个参数用来指明所使能定时器的中断类型。

第三个参数为TIMx中断的状态，该参数可取值为ENABLE(使能）或者DISABLE(失能)

9.2.2 定时器标准外设库配置过程

Setp 4

TIM2中断优先级设置

```
NVIC_InitTypeDef NVIC_InitStructure;
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);
```

9.2.2 定时器标准外设库配置过程

Setp 5

使能TIM2

使用TIM_Cmd()函数使能相应的定时器，其函数原型为：

```
void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState
NewState) ;
```

第一个参数TIMx是具体定时器，x可为2~4

第二个参数为TIMx的状态，该参数可取值为ENABLE（使能）或者DISABLE（失能）

例如，使能定时器TIM2的更新中断，配置如下：

```
TIM_ITConfig(TIM2,TIM_IT_Update,ENABLE);
```

9.2.2 定时器标准外设库配置过程

Setp 6

编写中断服务函数

STM32标准外设库函数中用来读取中断状态的库函数为：

```
ITStatus TIM_GetITStatus(TIM_TypeDef* TIMx, uint16_t);
```

清除定时器相应中断标志位的函数为：

```
void TIM_ClearITPendingBit(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```

例如，定时器TIM2的中断服务函数，代码如下：

```
void TIM2_IRQHandler(void)
{
   if(TIM_GetITStatus(TIM2,TIM_IT_Update) != RESET)
        //检查TIM2更新中断发生与否
   {        //LED的状态翻转

GPIO_WriteBit(GPIOB,GPIO_Pin_5,(BitAction)((1-GPIO_ReadOutputDataBit(GPIOB,GPIO_Pin_5))));

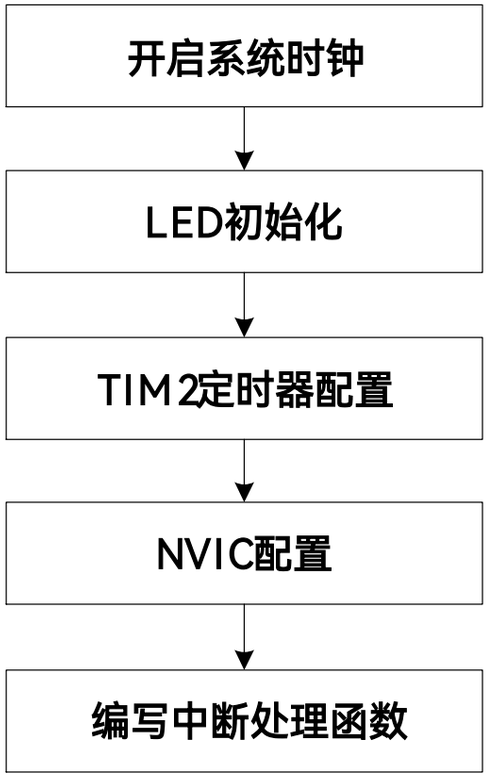
TIM_ClearITPendingBit(TIM2,TIM_IT_Update);
            //清除 TIM2 更新中断标志
   }
```

9.2.3 定时器标准外设库应用实例

硬件设计

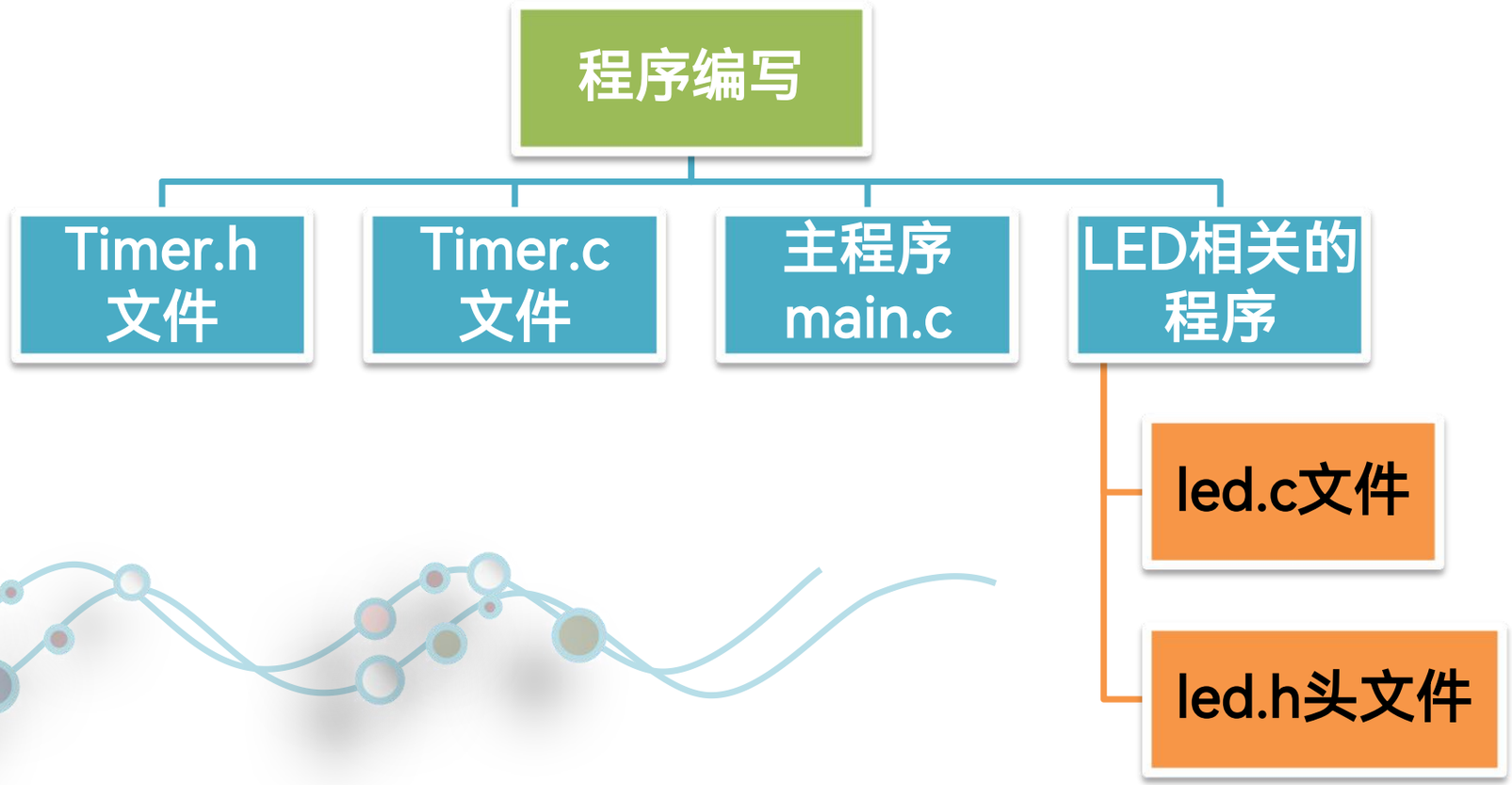
利用通用定时器TIM2实现LED精确延时1s闪烁，硬件设计上只需要一个LED灯通过串联一个限流电阻连接到STM32F103微控制器的PB5引脚即可。

软件设计



在系统初始化即开启系统时钟后，要对LED灯进行初始化操作，同时还需要配置通用定时器TIM2的相关参数，配置NVIC中断，最后编写中断处理函数。

9.2.3 定时器标准外设库应用实例



9.2.3 定时器标准外设库应用实例

1. 新建Timer.h文件，代码如下

```
#ifndef __TIMER_H
#define __TIMER_H
#include "stm32f10x.h"

void Timer_Init(void);
void TIM2_NVIC_Configuration(void);

#endif
```

9.2.3 定时器标准外设库应用实例

2．新建Timer.c文件，代码如下

```
#include "Timer.h"
void TIM2_NVIC_Configuration(void)
{       // NVIC配置
   NVIC_InitTypeDef NVIC_InitStructure;
   NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
       //NVIC中断分组
   NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
       //设置中断通道为定时器TIM2中断

NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority
= 0;
       //抢占优先级为0
   NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
        //响应优先级为1
   NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
       //使能引脚为中断源
   NVIC_Init(&NVIC_InitStructure);  //NVIC初始化
}
```

```
  //定时器TIM2初始化
void Timer_Init(void)
{    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
   RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2,ENABLE);
       //开启定时器TIM2的时钟
   TIM_TimeBaseStructure.TIM_Period = (36000-1);
       //设置自动重载计数周期值
   TIM_TimeBaseStructure.TIM_Prescaler = (2000-1);
       //设置分频系数
   TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
       //设置时钟分频因子
   TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
        //设置为向上计数方式
   TIM_TimeBaseInit(TIM2,&TIM_TimeBaseStructure);
       //初始化时钟TIM2
   TIM_ClearFlag(TIM2,TIM_FLAG_Update);
        //清中断，以免系统启动中断后立即产生中断
   TIM_ITConfig(TIM2,TIM_IT_Update,ENABLE);
        //使能定时器TIM2的更新中断
   TIM_Cmd(TIM2,ENABLE); //使能TIM2定时器
}
```

9.2.3 定时器标准外设库应用实例

3. 主程序main.c

```
#include "stm32f10x.h"
#include "led.h"
#include "Timer.h"

int main(void)
{
   LED_Init();//LED初始化
   Timer_Init();//定时器初始化
   TIM2_NVIC_Configuration(); //NVIC配置
   while(1)
   { } //为空，什么都不做
}
```

```
        //定时器TIM2中断处理函数
void TIM2_IRQHandler(void)
{
   if(TIM_GetITStatus(TIM2,TIM_IT_Update) !=
RESET)
   {
            // 将LED的状态翻转

GPIO_WriteBit(GPIOB,GPIO_Pin_5,(BitAction)((1-
GPIO_ReadOutputDataBit(GPIOB,GPIO_Pin_5))));
      TIM_ClearITPendingBit(TIM2,TIM_IT_Update);
            //清除 TIM2 更新中断标志
   }
}
```

9.2.3 定时器标准外设库应用实例

4．LED相关的程序

led.c文件，主要是LED灯的初始化，代码如下

```
#include "led.h"
void LED_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStructure;
RCC_APB2PeriphClockCmd(RCC_APB2Periph_G
PIOB,ENABLE);
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
  GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_Out_PP;
  GPIO_InitStructure.GPIO_Speed =
GPIO_Speed_50MHz;
  GPIO_Init(GPIOB,&GPIO_InitStructure);
}
```

led.h头文件，代码如下：

```
#ifndef __LED_H
#define __LED_H

#include "stm32f10x.h"
void LED_Init(void);

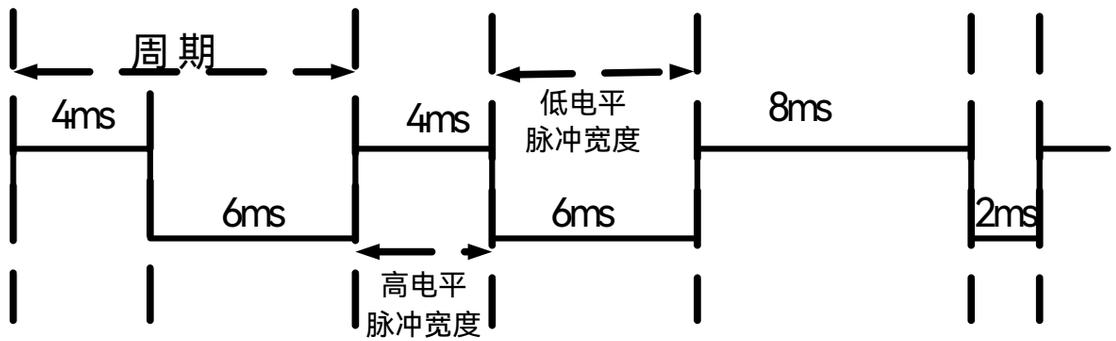
#endif
```

9.3 PWM

PWM(Pulse Width Modulation，脉冲宽度调制）是一种利用脉冲宽度即占空比实现对模拟信号进行控制的技术，即是对模拟信号电平进行数字表示的方法。

广泛应用于电力电子技术中，比如PWM控制技术在逆变电路中的应用；

PWM还应用于直流电机调速，如变频空调的交直流变频调速，除实现调速外，还具有节能等特性。

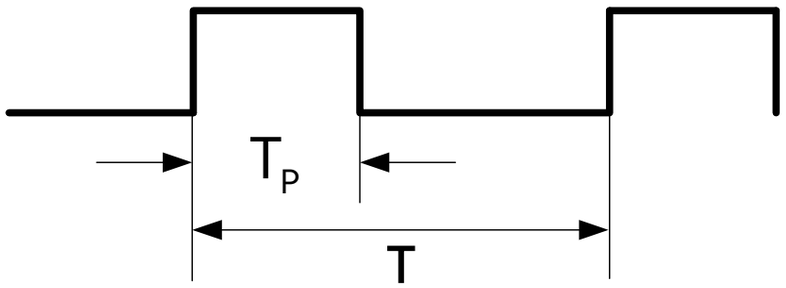


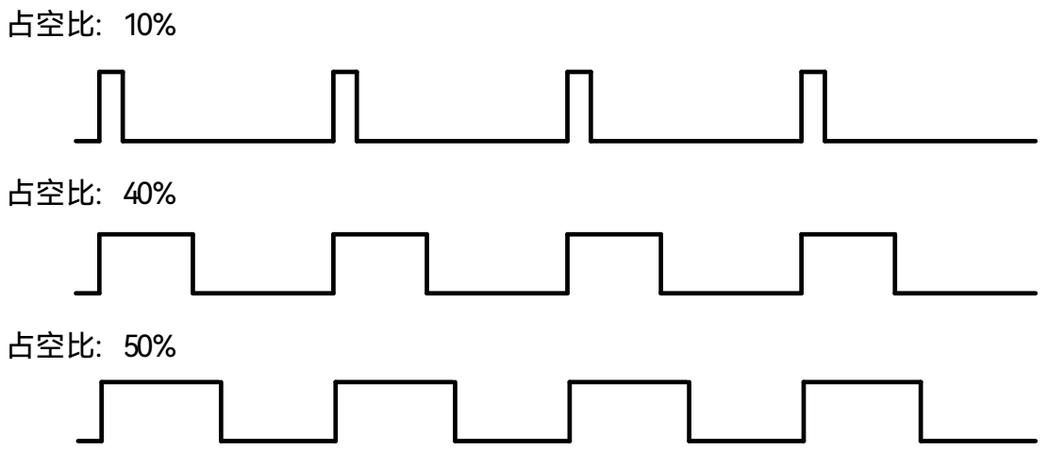
周期为10ms(频率为100Hz)的PWM波形

9.3 PWM

占空比(Duty Cycle)，是指在一个周期内，高电平时间占整个信号周期的百分比，即高电平时间与周期的比值：

占空比=Tp/T





不同占空比的PWM波形

9.3 PWM



9.3.1 PWM的工作原理



9.3.2 PWM标准外设库输出配置过程

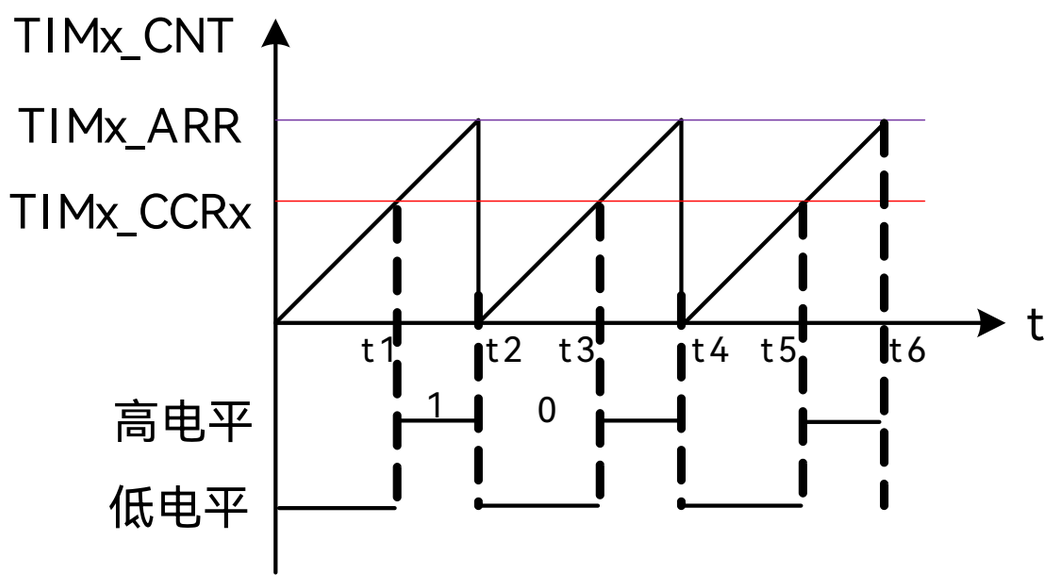


9.3.3 PWM标准外设库应用实例

9.3.1 STM32定时器PWM的工作原理

- STM32的定时器除了TIM6和TIM7，其他定时器都可以用来产生PWM输出；
- 高级定时器TIM1和TIM8可以同时产生多达7路的PWM输出；
- 通用定时器能同时产生多达4路的PWM输出；
- STM32中每个定时器有4个输入通道：TIMx_CH1~TIMx_CH4；
- 每个通道对应1个捕获/比较寄存器TIMx_CRRx，将寄存器值和计数器值相比较，通过比较结果输出高低电平，从而得到PWM信号；
- 脉冲宽度调制模式可以产生一个由TIMx_ARR寄存器确定频率、由TIMx_CCRx寄存器确定占空比的信号。

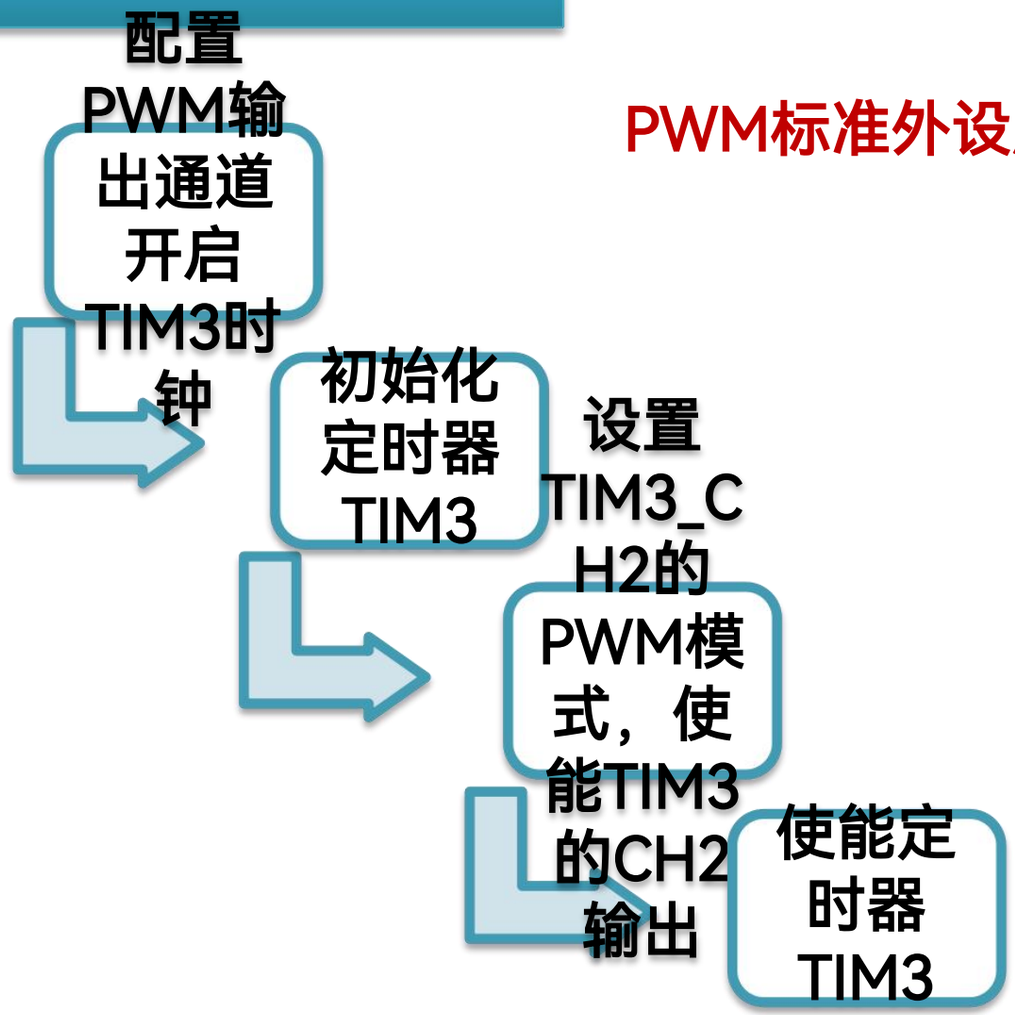
9.3.1 PWM的工作原理



- 在PWM的一个周期内，定时器从0开始向上计数，在0-t1时间段，定时器计数器TIMx_CNT值小于TIMx_CCRx值，输出低电平；
- 在t1-t2时间段，定时器计数器TIMx_CNT值大于TIMx_CCRx值，输出高电平；
- 当定时器计数器的值TIMx_CNT达到ARR时，定时器溢出，重新从0开始向上计数，如此循环。

9.3.2 PWM标准外设库输出配置过程

PWM标准外设库输出配置步骤



9.3.2 PWM标准外设库输出配置过程

1．配置PWM输出通道，开启TIM3时钟

STM32规定了具体的引脚作为PWM输出引脚，选择不同的引脚时，还必须使用重定向功能(Remap）。定时器TIM1的引脚复用功能映像：

复用功能	TIM1_REMAP[1:0] = 00 (没有重映像)	TIM1_REMAP[1:0] = 01 (部分重映像)	TIM1_REMAP[1:0] = 11 (完全重映像) (1)
TIM1_ETR	PA12		PE7
TIM1_CH1	PA8		PE9
TIM1__CH2	PA9		PE11
TIM1_CH3	PA10		PE13
TIM1_CH4	PA11		PE14
TIM1_BKIN	PB12(2)	PA6	PE15
TIM1_CH1N	PB13(2)	PA7	PE8
TIM1_CH2N	PB14(2)	PB0	PE10
TIM1_CH3N	PB15(2)	PB1	PE12

9.3.2 PWM标准外设库输出配置过程

1．配置PWM输出通道，开启TIM3时钟

定时器TIM2的引脚复用功能映像：

复用功能	TIM2_REMAP[1:0]=00 (没有重映像)	TIM1_REMAP[1:0]=01 (部分重映像)	TIM1_REMAP[1:0]=10 (部分重映像)(1)	TIM1_REMAP[1:0]=11 (完全重映像)(1)
TIM2_CH1_ETR(2)	PA0	PA15	PA0	PA15
TIM2_CH2	PA1	PB3	PA1	PB3
TIM2_CH3	PA2		PB10	
TIM2_CH4				

注：(1) 重映像不适用于36引脚的封装；(2) TIM2 _ETR和TIM2_CH1共用一个引脚，但不能同时使用。

9.3.2 PWM标准外设库输出配置过程

1．配置PWM输出通道，开启TIM3时钟

定时器TIM3的引脚复用功能映像：

复用功能	TIM3_REMAP[1:0] = 00 (没有重映像)	TIM3_REMAP[1:0] = 10 (部分重映像)	TIM3_REMAP[1:0] = 11 (完全重映像) (1)
TIM3_CH1	PA6	PB4	PC6
TIM3_CH2	PA7	PB5	PC7
TIM3_CH3	PB0		PC8
TIM3_CH4	PB1		PC9

注：(1) 重映像只适用于64、100和144引脚的封装。

9.3.2 PWM标准外设库输出配置过程

1．配置PWM输出通道，开启TIM3时钟

- 使用定时器3的通道2（TIM3_CH2）作为PWM的输出引脚；
- 对PB5引脚进行配置，配置PB5为复用推挽输出模式（GPIO_Mode_AF_PP）

```
GPIO_InitTypeDef GPIO_InitStructure;
  //定义结构体
  //使能GPIO外设和AFIO复用功能模块时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RC
C_APB2Periph_AFIO,ENABLE);
  //选择TIM3部分重映像
GPIO_PinRemapConfig(GPIO_PartialRemap_TIM3,
ENABLE);
  //选择定时器3的通道2作为PWM的输出引脚
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5; //TIM_CH2
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
  //复用推挽输出模式
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOB, &GPIO_InitStructure);//初始化PB5引脚
```

9.3.2 PWM标准外设库输出配置过程

2．初始化定时器TIM3

- 设置预分频系数PSC、自动重载计数值ARR、计数模式、时钟分割等。

```
TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
    //定义初始化结构体
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
    //使能TIM3时钟
    //初始化TIM3
TIM_TimeBaseStructure.TIM_Period = arr; //设置自动重载值
TIM_TimeBaseStructure.TIM_Prescaler =psc;
    //设置用来作为TIM3时钟频率的预分频值
TIM_TimeBaseStructure.TIM_ClockDivision = 0;
   //设置时钟分割TDTS = Tck_tim
TIM_TimeBaseStructure.TIM_CounterMode =
TIM_CounterMode_Up;
    //TIM向上计数模式
TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);
   //初始化TIM3
```

9.3.2 PWM标准外设库输出配置过程

3．设置TIM3_CH2的PWM模式，使能TIM3的CH2输出

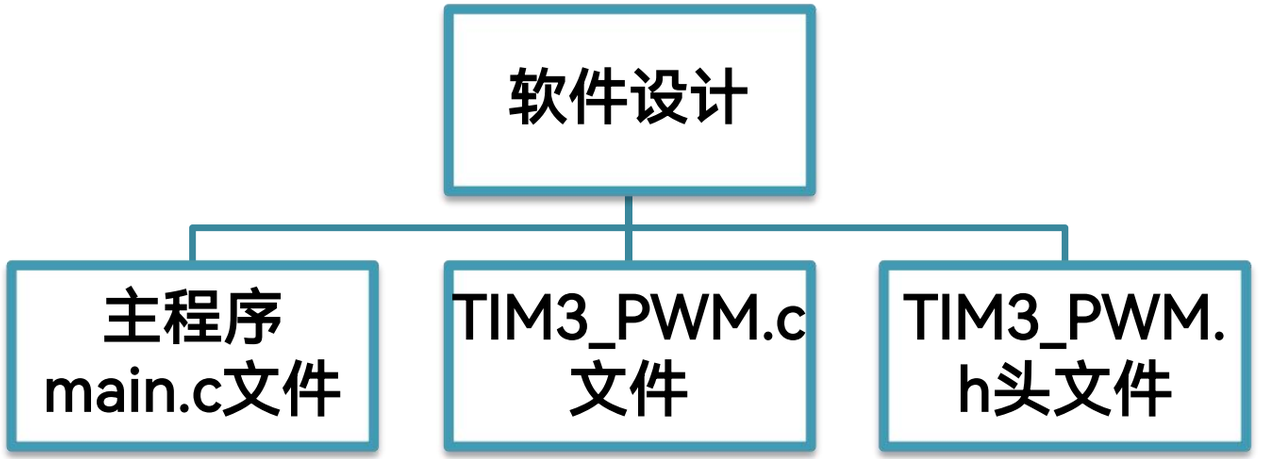
```
    //初始化定时器TIM3通道2 PWM模式
TIM_OCInitTypeDef  TIM_OCInitStructure;
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
    //设置定时器模式：PWM2模式
 TIM_OCInitStructure.TIM_OutputState =
TIM_OutputState_Enable;
    //比较输出使能
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    //设置输出极性
TIM_OC2Init(TIM3, &TIM_OCInitStructure);  //初始化TIM3 OC2
TIM_OC2PreloadConfig(TIM3, TIM_OCPreload_Enable);
    //使能TIM3在CCR2上的预装载寄存器
```

4．使能定时器TIM3

```
TIM_Cmd(TIM3, ENABLE);  //使能TIM3
```

9.3.3 PWM标准外设库应用实例

功能描述：基于PWM的呼吸灯。使用通用定时器TIM3产生PWM信号，输出占空比可变的PWM波，用于驱动LED灯，使LED亮度由暗变亮，又从亮变暗，如此循环。。



9.3.3 PWM标准外设库应用实例

1．主程序main.c文件

```
#include "stm32f10x.h"
#include "TIM3_PWM.h"
//延迟函数Delay()
void Delay(__IO u32 nCount)
{   for(;nCount !=0;nCount --); }

int main(void)
{
   u16 PWM_Val=0;
      //定义变量PWM_Val为PWM的占空比
   u8 pwd=1; //定义一个变量，用于判断
  TIM3_PWM_Init(899,0);
      //不分频，PWM频率
=72000000/900=80KHz
  while(1)
 {
      Delay(10000);
      if(pwd)PWM_Val++;
        //如果pwd为真，则占空比逐渐增加
 else PWM_Val--; //否则逐渐减小
 if(PWM_Val>300) pwd =0;
      //如果占空比大于300，则pwd设置为0
 if(PWM_Val==0) pwd =1;
      //如果占空比为0，将标识pwd设置为1
  TIM_SetCompare2(TIM3,PWM_Val); //修改占空比
  }
}
     //定时器TIM3的中断服务程序
void TIM3_IRQHandler(void)
{
   if (TIM_GetITStatus(TIM3, TIM_IT_Update) !=
RESET)
  {

      GPIO_WriteBit(GPIOB,GPIO_Pin_5,(BitAction)
      ((1-
      GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_5)));
       TIM_ClearITPendingBit(TIM3, TIM_IT_Update);
```

9.3.3 PWM标准外设库应用实例

2．TIM3_PWM.c文件

```
#include "TIM3_PWM.h"
#include "stm32f10x.h"
void TIM3_PWM_Init(u16 arr,u16 psc)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
    TIM_OCInitTypeDef  TIM_OCInitStructure;
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3,
ENABLE);
        //使能定时器TIM3的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC
    _APB2Periph_AFIO, ENABLE);
        //使能GPIOB和AFIO时钟
    GPIO_PinRemapConfig(GPIO_PartialRemap_TIM3,ENABL
E);
        //TIM3部分重映射TIM3_CH2->PB5
        //设置PB5引脚为复用输出引脚
        // TIM3 CH2 输出PWM脉冲波形
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5; //PB5
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
        // PB5引脚复用为TIM3_CH2
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

```
        // TIM3初始化
    TIM_TimeBaseStructure.TIM_Period = arr; //设置自动重载
值
    TIM_TimeBaseStructure.TIM_Prescaler =psc;
        //设置用来作为TIM3时钟频率的预分频值
    TIM_TimeBaseStructure.TIM_ClockDivision = 0;
        //设置时钟分割TDTS = Tck_tim
    TIM_TimeBaseStructure.TIM_CounterMode =
    TIM_CounterMode_Up; //TIM向上计数模式
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);
    //初始化TIM3；初始化定时器TIM3通道2 PWM模式
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
        //设置定时器模式：PWM2模式
     TIM_OCInitStructure.TIM_OutputState =
    TIM_OutputState_Enable; //比较输出使能
    TIM_OCInitStructure.TIM_OCPolarity =
TIM_OCPolarity_High;
        //设置输出极性
    TIM_OC2Init(TIM3, &TIM_OCInitStructure);  //初始化TIM3
OC2
    TIM_OC2PreloadConfig(TIM3, TIM_OCPreload_Enable);
        //使能TIM3在CCR2上的预装载寄存器
```

9.3.3 PWM标准外设库应用实例

3．TIM3_PWM.h头文件

```
#ifndef __TIMER_H
#define __TIMER_H
#include "stm32f10x.h"
void TIM3_PWM_Init(u16 arr,u16 psc);
#endif
```

9.4 SysTick定时器

- Cortex-M3内核中的SysTick定时器，是一个24位的从重载值向下递减到0的计数器，是NVIC的一部分，根植于NVIC;
- 常用于精确定时，为操作系统提供必要的时钟节拍，为RTOS的任务调度提供-

SysTick定时器一共有四个寄存器，分别是

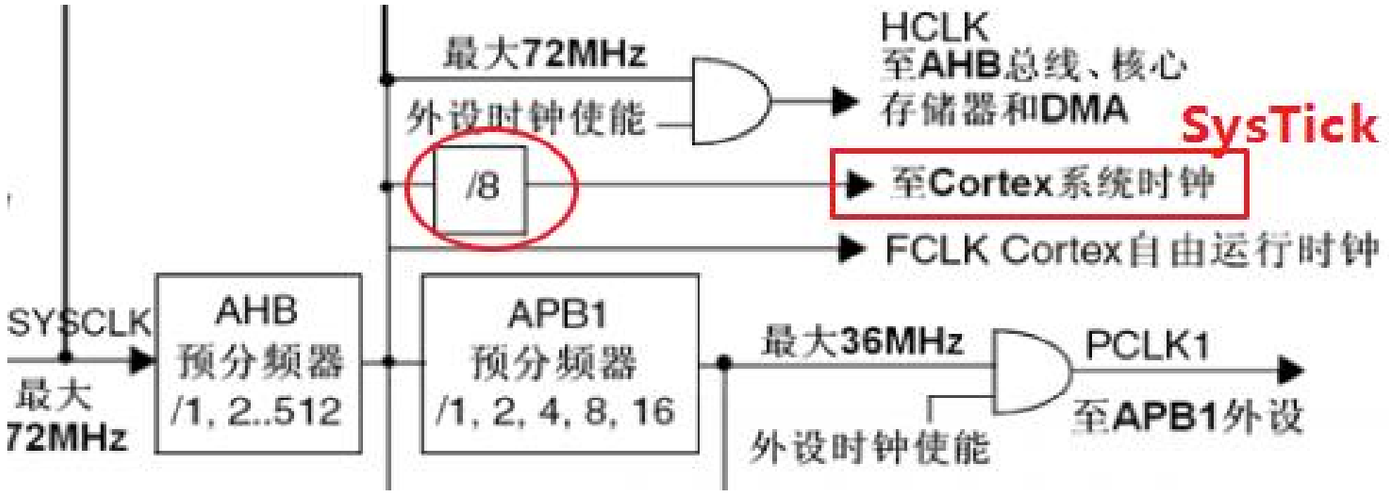
- SysTick控制及状态寄存器CTRL
- SysTick重装载数值寄存器LOAD
- SysTick当前数值寄存器VAL

LIB。

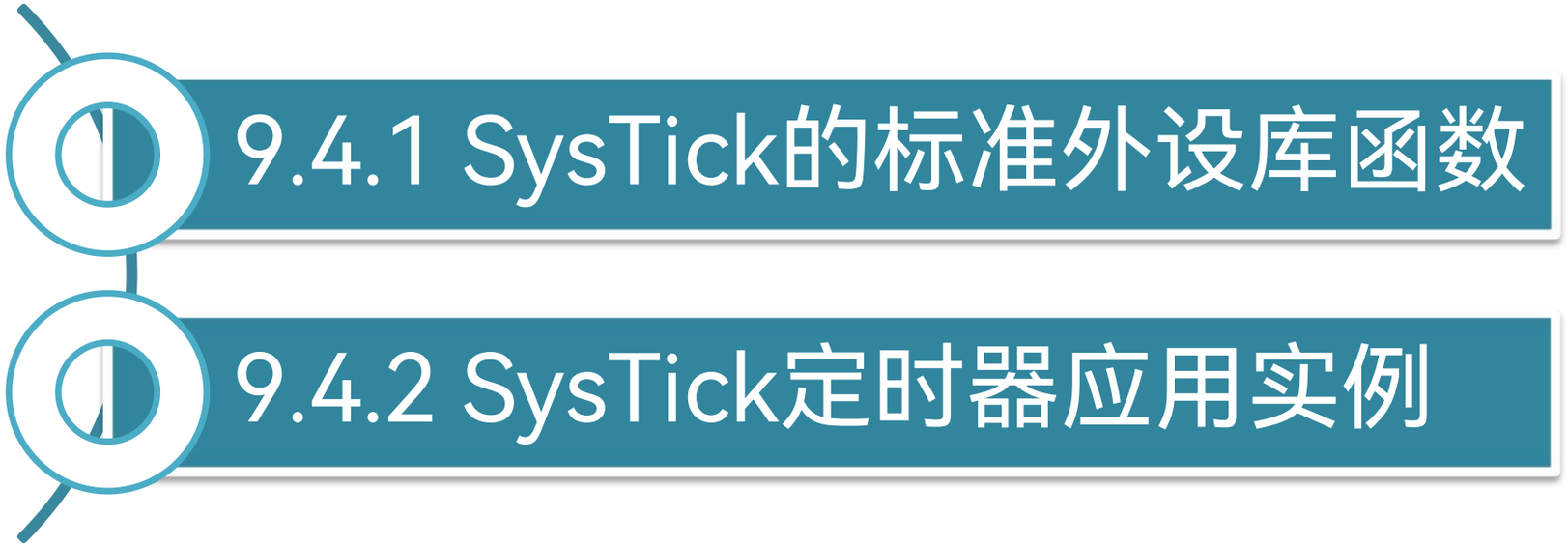
Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	STK_CTRL	Reserved															COUNTFLAG	Reserved													CLKSOURCE	TICK INT	EN ABLE
	Reset Value																0														1	0	0
0x04	STK_LOAD	Reserved								RELOAD[23:0]																							
	Reset Value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	STK_VAL	Reserved								CURRENT[23:0]																							
	Reset Value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	STK_CALIB	Reserved								TENMS[23:0]																							
	Reset Value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

9.4 SysTick定时器

- SysTick定时器的时钟源可以是内部时钟(FCLK)或者是外部时钟，系统默认的SysTick定时器是由AHB时钟(HCLK)8分频得到的，即SysTick的频率为9MHz。SysTick定时器从设定的初值计数到0时，会自动重装初值继续计数，同时触发中断，因此，只需确定计数的次数就可以精确得到延迟时间。



9.4 SysTick定时器



9.4.1 SysTick的标准外设库函数

1．SysTick相关寄存器的定义在core_cm3.h文件中

```
typedef struct
{
  __IO uint32_t CTRL;     /*!< Offset: 0x00  SysTick Control and Status
Register */
  __IO uint32_t LOAD;    /*!< Offset: 0x04  SysTick Reload Value Register
*/
  __IO uint32_t VAL;      /*!< Offset: 0x08  SysTick Current Value Register
*/
  __I  uint32_t CALIB;   /*!< Offset: 0x0C  SysTick Calibration Register       */
} SysTick_Type;
```

9.4.1 SysTick的标准外设库函数

2．SysTick的库函数

1）SysTick_Config(uint32_t ticks)，定义在core_cm3.h头文件中，代码如下：

```
static __INLINE uint32_t SysTick_Config(uint32_t ticks)
{
 if (ticks > SysTick_LOAD_RELOAD_Msk)  return (1);  /* 若重装载值超过24位，失败返回1 */
 SysTick->LOAD  = (ticks & SysTick_LOAD_RELOAD_Msk) - 1; /* 设置重装载寄存器*/
 NVIC_SetPriority (SysTick_IRQn, (1<<__NVIC_PRIO_BITS) - 1);  /* 设置系统中断的优先级 */
 SysTick->VAL   = 0;          /*装载SysTick的计数值(当前计数值清0）*/
 SysTick->CTRL  = SysTick_CTRL_CLKSOURCE_Msk |
                  SysTick_CTRL_TICKINT_Msk   |
                  SysTick_CTRL_ENABLE_Msk;  /* 使能SysTick中断和SysTick定时器*/
 return (0);                  /* 成功，返回0*/
}
```

9.4.1 SysTick的标准外设库函数

2．SysTick的库函数

2）SysTick_CLKSourceConfig(uint32_t SysTick_CLKSource)，在misc.c文件中定义。

```
void SysTick_CLKSourceConfig(uint32_t SysTick_CLKSource)
{
  /* Check the parameters */
  assert_param(IS_SYSTICK_CLK_SOURCE(SysTick_CLKSource));
  if (SysTick_CLKSource == SysTick_CLKSource_HCLK)
  {   SysTick->CTRL |= SysTick_CLKSource_HCLK;
  }
  else
  {   SysTick->CTRL &= SysTick_CLKSource_HCLK_Div8;
  }
}
```

此函数用于选择SysTick定时器的时钟源，有两个选项：SysTick_CLKSource_HCLK和SysTick_CLKSource_HCLK_Div8。

AHB时钟，不分频，则SysTick的时钟频率为72MHz，设重装载值为72000，则延时时间为72000*(1/72000 000Hz)=1ms。

为AHB的8分频，则SysTick的时钟频率为9MHz(72MHz/8=9MHz)。设重装载值为9000，则延时时间为9000*(1/9000 000Hz）=1ms。

9.4.2 SysTick定时器应用实例

基于STM32F10x v3.5.0标准外设库操作SysTick定时器实现精确延时1ms。



初始化SysTick:

- 利用SysTick_Config(uint32_t ticks)函数配置SysTick的重装载初值;
- 利用SysTick_CLKSourceConfig(uint32_t SysTick_CLKSource)函数选择SysTick的时钟源

编写delay延迟函数

编写SysTick中断处理函数

9.4.2 SysTick定时器应用实例

软件代码实现

- 新建systick.h头文件
- 新建systick.c文件
- 新建mian.c文件
- 编写中断处理函数
- 编写LED相关的代码

```
#ifndef __SYSTICK_H
#define __SYSTICK_H
#include "stm32f10x.h"

void SysTick_Init(void);
void Delay_ms(uint16_t nCount);

#endif
```

9.4.2 SysTick定时器应用实例

软件代码实现

- 新建systick.h头文件
- 新建systick.c文件
- 新建mian.c文件
- 编写中断处理函数
- 编写LED相关的代码

```
#include "systick.h"
__IO uint32_t TimingDelay;
 //定义一个全局变量，注意类型为volatile的
void SysTick_Init(void)
{    SysTick_Config(9000);
//9000为重装载值，SysTick时钟频率为9MHz，定时时间1
ms
SysTick_CLKSourceConfig(SysTick_CLKSource_HCL
K_Div8);
}
void Delay_ms(uint16_t nCount)
{              TimingDelay = nCount;
               while(TimingDelay != 0);
```

9.4.2 SysTick定时器应用实例

软件代码实现

- 新建systick.h头文件
- 新建systick.c文件
- 新建mian.c文件
- 编写中断处理函数
- 编写LED相关的代码

```
#include "stm32f10x.h"
#include "led.h"
#include "systick.h"
int main(void)
{
   LED_Init();
   SysTick_Init();
   while(1)
   {
      GPIO_SetBits(GPIOB,GPIO_Pin_5);
      Delay_ms(1000);  // 延时 1000 * (1/1000) = 1
         s
      GPIO_ResetBits(GPIOB,GPIO_Pin_5);
      Delay_ms(1000);
   }
}
```

9.4.2 SysTick定时器应用实例

软件代码实现

- 新建systick.h头文件
- 新建systick.c文件
- 新建mian.c文件
- 编写中断处理函数
- 编写LED相关的代码

```
extern __IO uint32_t TimingDelay;
// 添加外部的声明，告诉编译器这是在外部定义的一个变量
void SysTick_Handler(void)
{
        if(TimingDelay != 0x00)
        {
                TimingDelay --;
        }
}
```

9.4.2 SysTick定时器应用实例

软件代码实现

- 新建systick.h头文件
- 新建systick.c文件
- 新建mian.c文件
- 编写中断处理函数
- 编写LED相关的代码

led.c

```
#include "led.h"
void LED_Init(void)
{
   GPIO_InitTypeDef GPIO_InitStructure;

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIO
B, ENABLE);
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
  GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_Out_PP;
  GPIO_InitStructure.GPIO_Speed =
GPIO_Speed_50MHz;
  GPIO_Init(GPIOB,&GPIO_InitStructure);
```

9.4.2 SysTick定时器应用实例

软件代码实现

- 新建systick.h头文件
- 新建systick.c文件
- 新建mian.c文件
- 编写中断处理函数
- 编写LED相关的代码

编译下载，可以观察实验现象：LED灯精确延迟1s循环闪烁。

led.h

```
#ifndef __LED_H
#define __LED_H

#include "stm32f10x.h"
void LED_Init(void);

#endif
```

9.4.2 SysTick定时器应用实例

HAL库有一个延时函数HAL_Delay(uint32_t Delay)，提供毫秒级的延时

```
void HAL_Delay(uint32_t Delay)
{
  uint32_t  tickstart = HAL_GetTick();
  uint32_t  wait = Delay;
  /* Add a freq to guarantee minimum
wait */
  if (wait < HAL_MAX_DELAY)
  {
    wait += (uint32_t)(uwTickFreq);
  }
  while ((HAL_GetTick() - tickstart) < wait)
  {
  }
}
```

9.4.2 SysTick定时器应用实例

HAL_Delay( )函数通过调用HAL_GetTick( )函数，返回uwTick值

```
uint32_t  HAL_GetTick(void)
{  return  uwTick;
}
```

通过单击鼠标右键“Go To Definition of ‘ ’”跳转到uwTick

```
__IO uint32_t  uwTick;
__weak void HAL_IncTick(void)
{  uwTick += uwTickFreq;  }
```

uwTickFreq的定义：uwTickFreq默认为系统的TICK，频率为1kHz

```
#define  HAL_MAX_DELAY  0xFFFFFFFFU
HAL_TickFreqTypeDef  uwTickFreq =
HAL_TICK_FREQ_DEFAULT;  /* 1KHz */
HAL_TICK_FREQ_DEFAULT   =
HAL_TICK_FREQ_1KHZ
```

9.5 定时器模块的HAL库接口函数及应用

目 录


9.5.1定时器HAL库接口函数


9.5.2定时器HAL库应用实例

9.5.1 定时器HAL库接口函数

定时器模块的HAL库常用接口函数可分为七大类：

类型	函数及功能描述
基本定时器功能函数	HAL_TIM_Base_Init(TIM_HandleTypeDef *htim); 功能描述：基本定时器初始化函数
	void HAL_TIM_Base_MspInit(TIM_HandleTypeDef *htim); 功能描述：基本定时器硬件初始化配置函数，该函数被HAL库内部调用
定时器输出比较功能函数	HAL_TIM_OC_Init(TIM_HandleTypeDef *htim); 功能描述：定时器输出比较的初始化函数
	HAL_TIM_OC_Start(TIM_HandleTypeDef *htim, uint32_t Channel); 功能描述：定时器输出比较轮询方式的启动函数
	HAL_TIM_OC_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel); 功能描述：定时器输出比较中断方式的启动函数
	HAL_TIM_OC_Start_DMA(TIM_HandleTypeDef *htim, uint32_t Channel, uint32_t *pData, uint16_t Length); 功能描述：定时器输出比较DMA方式的启动函数
定时器PWM函数	HAL_TIM_PWM_Init(TIM_HandleTypeDef *htim); 功能描述：定时器PWM初始化函数
	HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel); 功能描述：启动定时器PWM轮询方式的函数
	HAL_TIM_PWM_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel); 功能描述：启动定时器PWM中断方式的函数
	HAL_TIM_PWM_Start_DMA(TIM_HandleTypeDef *htim, uint32_t Channel, uint32_t *pData, uint16_t Length); 功能描述：启动定时器PWM的DMA方式的函数

9.5.1 定时器HAL库接口函数

定时器模块的HAL库常用接口函数可分为七大类：

类型	函数及功能描述
定时器输入捕获函数	HAL_TIM_IC_Init(TIM_HandleTypeDef *htim); 功能描述：定时器输入捕获初始化函数
	HAL_TIM_IC_Start(TIM_HandleTypeDef *htim, uint32_t Channel); 功能描述：启动定时器输入捕获轮询方式的函数
	HAL_TIM_IC_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel); 功能描述：启动定时器输入捕获中断方式的函数
	HAL_TIM_IC_Start_DMA(TIM_HandleTypeDef *htim, uint32_t Channel, uint32_t *pData, uint16_t Length); 功能描述：启动定时器输入捕获的DMA方式的函数
定时器中断及回调函数	void HAL_TIM_IRQHandler(TIM_HandleTypeDef *htim); 功能描述：定时器中断处理函数
	void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim); 功能描述：定时器定时/更新中断回调函数，用户在该函数内编写实际的中断处理程序
	void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim); 功能描述：定时器输入捕获回调函数
	void HAL_TIM_PWM_PulseFinishedCallback(TIM_HandleTypeDef *htim); 功能描述：定时器PWM结束回调函数

9.5.1 定时器HAL库接口函数

定时器模块的HAL库常用接口函数可分为七大类：

类型	函数及功能描述
定时器功能配置函数	HAL_TIM_OC_ConfigChannel(TIM_HandleTypeDef *htim, TIM_OC_InitTypeDef* sConfig, uint32_t Channel); 功能描述：定时器输出比较配置函数
	HAL_TIM_PWM_ConfigChannel(TIM_HandleTypeDef *htim, TIM_OC_InitTypeDef* sConfig, uint32_t Channel); 功能描述：定时器PWM配置函数
	HAL_TIM_IC_ConfigChannel(TIM_HandleTypeDef *htim, TIM_IC_InitTypeDef* sConfig, uint32_t Channel); 功能描述：定时器输入捕获配置函数
定时器状态函数	HAL_TIM_OC_GetState(TIM_HandleTypeDef *htim); 功能描述：获取定时器输出比较的状态函数
	HAL_TIM_PWM_GetState(TIM_HandleTypeDef *htim); 功能描述：获取定时器PWM的状态函数
	HAL_TIM_IC_GetState(TIM_HandleTypeDef *htim); 功能描述：获取定时器输入捕获的状态函数

9.5.1 定时器HAL库接口函数

1．定时器初始化配置函数

HAL_TIM_Base_Init()

```
HAL_StatusTypeDef
HAL_TIM_Base_Init(TIM_HandleTypeDef *htim)
{
   /* 检测htim句柄是否有效 */
  if(htim == NULL)
  {
    return HAL_ERROR;
  }
   /* 采用断言方式检测参数是否有效 */
  assert_param(IS_TIM_INSTANCE(htim-
>Instance));
  assert_param(IS_TIM_COUNTER_MODE(htim-
>Init.CounterMode));
```

```
HAL_StatusTypeDef
HAL_TIM_Base_Init(TIM_HandleTypeDef *htim)
{
   /* 检测htim句柄是否有效 */
  if(htim == NULL)
  {
    return HAL_ERROR;
  }
   /* 采用断言方式检测参数是否有效 */
  assert_param(IS_TIM_INSTANCE(htim-
>Instance));
  assert_param(IS_TIM_COUNTER_MODE(htim-
>Init.CounterMode));

assert_param(IS_TIM_CLOCKDIVISION_DIV(htim-
>Init.ClockDivision));
```

9.5.1 HAL库定时器接口函数

1．定时器初始化配置函数 HAL_TIM_Base_Init()

解析： 函数源码中主要涉及到两个重要的函数HAL_TIM_Base_MspInit(htim)和TIM_Base_SetConfig( )，HAL库每个外设模块都设计有单独的底层初始化回调函数。

```
if(htim->State == HAL_TIM_STATE_RESET)
 {
    /* 取消上锁*/
  htim->Lock = HAL_UNLOCKED;
    /*初始化底层硬件：GPIO、CLOCK、 NVIC */
  HAL_TIM_Base_MspInit(htim);
 }
  /*设置TIM状态 */
 htim->State= HAL_TIM_STATE_BUSY;
  //将状态设置为BUSY
  /* 配置TIM的基本参数 */
 TIM_Base_SetConfig(htim->Instance, &htim->Init);
  /* 初始化TIM 状态，TIM就绪 */
 htim->State= HAL_TIM_STATE_READY;
  //将状态设置为READY
 return HAL_OK;
}
```

9.5.1 HAL库定时器接口函数

HAL库在TIM_TypeDef的基础上封装了一个结构体TIM_HandleTypeDef（定时器句柄）

```
Typedef struct
{
  TIM_TypeDef     *Instance;  /*TIM寄存器的实例化*/
  TIM_Base_InitTypeDef    Init;
       /*配置定时器的基本参数 */
  HAL_TIM_ActiveChannel     Channel;   /*配置定时器通道
*/
  DMA_HandleTypeDef        *hdma[7];   /*配置DMA */
  HAL_LockTypeDef           Lock;        /*上锁*/
  __IO HAL_TIM_StateTypeDef   State;
       /*定时器操作状态*/
} TIM_HandleTypeDef;
```

9.5.1 HAL库定时器接口函数

TIM_HandleTypeDef结构体又包含了定时器初始化结构体

TIM_Base_InitTypeDef

```
typedef struct
{
  uint32_t  Prescaler; /*定时器预分频系数，其值范围在0x0000 ~ 0xFFFF之间 */
  uint32_t  CounterMode;    /*定时器计数模式，向上计数、向下计数和中心对齐模式
*/
  uint32_t  Period;          /*定时器定时周期，其值范围为0x0000 ~ 0xFFFF */
  uint32_t  ClockDivision;    /* 定时器时钟分频因子*/
  uint32_t  RepetitionCounter;  /*重复计数器，仅用于高级定时器TIM1和TIM8*/
  uint32_t  AutoReloadPreload;  /*定时器自动重装载值*/
} TIM_Base_InitTypeDef;
```

9.5.1 HAL库定时器接口函数

2．启动定时器函数

HAL_TIM_Base_Start()

解析： 该数只有一个输入参数，为TIM_HandleTypeDef类型的结构体指针变量，程序中主要是调用__HAL_TIM_ENABLE(htim)函数使能相应的定时器，该函数实质是一个宏定义：

#define __HAL_TIM_ENABLE(__HANDLE__) ((__HANDLE__)->Instance->CR1|=(TIM_CR1_CEN))

配置定时器CR1的使能位为“1”，返回HAL_OK，表示初始化成功。

```
HAL_StatusTypeDef
HAL_TIM_Base_Start(TIM_HandleTypeDef *htim)
{
  uint32_t  tmpsmcr;
   /* 使用断言，检测参数是否正确 */
  assert_param(IS_TIM_INSTANCE(htim->Instance));
   /*设置定时器的状态 */
  htim->State = HAL_TIM_STATE_BUSY;  /* 使能外设
*/
  tmpsmcr = htim->Instance->SMCR &
TIM_SMCR_SMS;
  if
(!IS_TIM_SLAVEMODE_TRIGGER_ENABLED(tmpsmcr))
  {    __HAL_TIM_ENABLE(htim);
  }
   /* 定时器就绪*/
  htim->State = HAL_TIM_STATE_READY;
   /* 返回HAL_OK，初始化成功*/
```

9.5.1 HAL库定时器接口函数

3．启动定时器中断模式定时函数HAL_TIM_Base_Start_IT()

解析： 该启动函数的核心代码为：

__HAL_TIM_ENABLE(htim)，跳转到该函数，可以发现该函数其实是一个宏定义：

```
#define __HAL_TIM_ENABLE(__HANDLE__) ((__HANDLE__)->Instance->CR1|=(TIM_CR1_CEN))
```

该宏定义的作用是将CR1的计数器使能位设置为“1”。

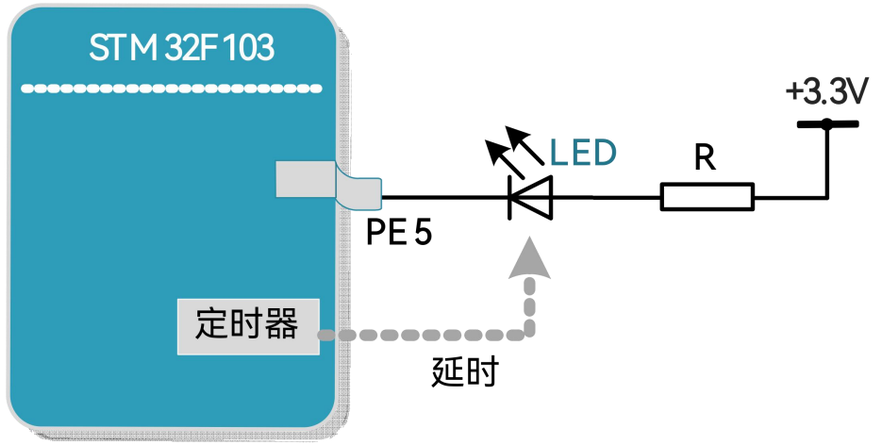
```
HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim)
{
  uint32_t tmpsmcr;
   /* 使用断言，进行参数状态检测*/
  assert_param(IS_TIM_INSTANCE(htim->Instance));
   /*使能定时器更新中断*/
  __HAL_TIM_ENABLE_IT(htim, TIM_IT_UPDATE);
   /* 使能外设*/
  tmpsmcr = htim->Instance->SMCR & TIM_SMCR_SMS;
  if (!IS_TIM_SLAVEMODE_TRIGGER_ENABLED(tmpsmcr))
  {   __HAL_TIM_ENABLE(htim);
  }
  /* 返回HAL_OK*/
  return HAL_OK;
```

9.5.2 定时器HAL库应用实例

功能

利用基于HAL库采用定时器TIM3产生精确延时1s

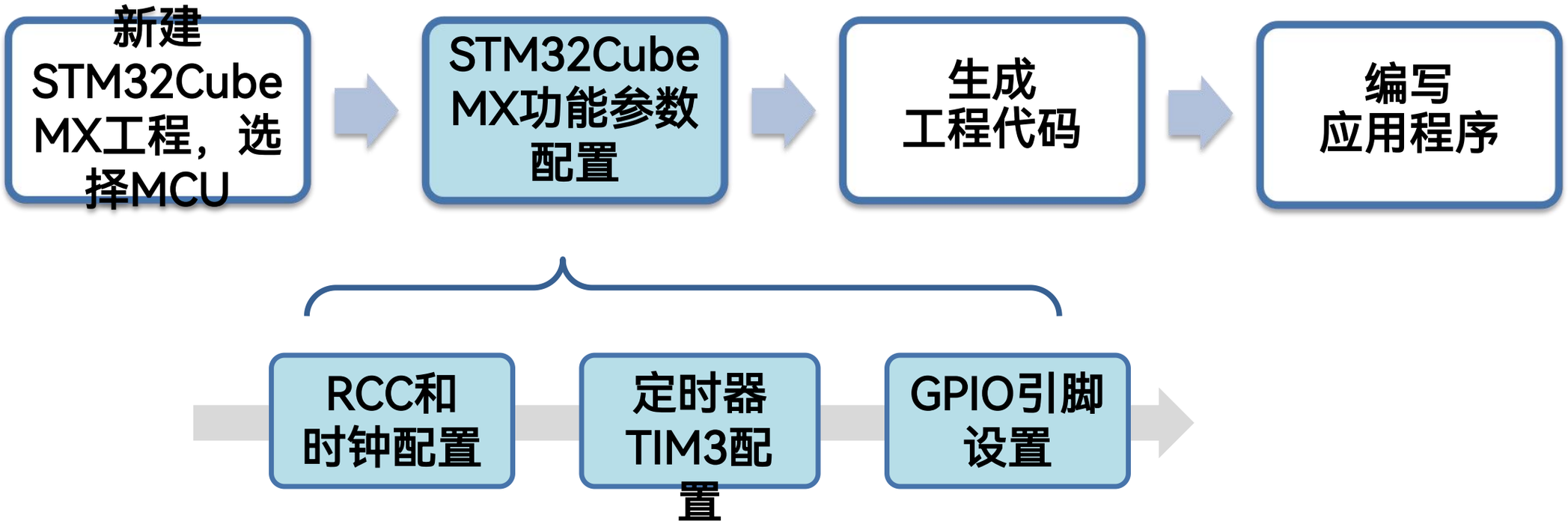
硬件设计



利用定时器TIM3产生1s的定时中断，通过LED指示灯进行状态显示，LED连接在STM32F103的PE5引脚上，低电平有效

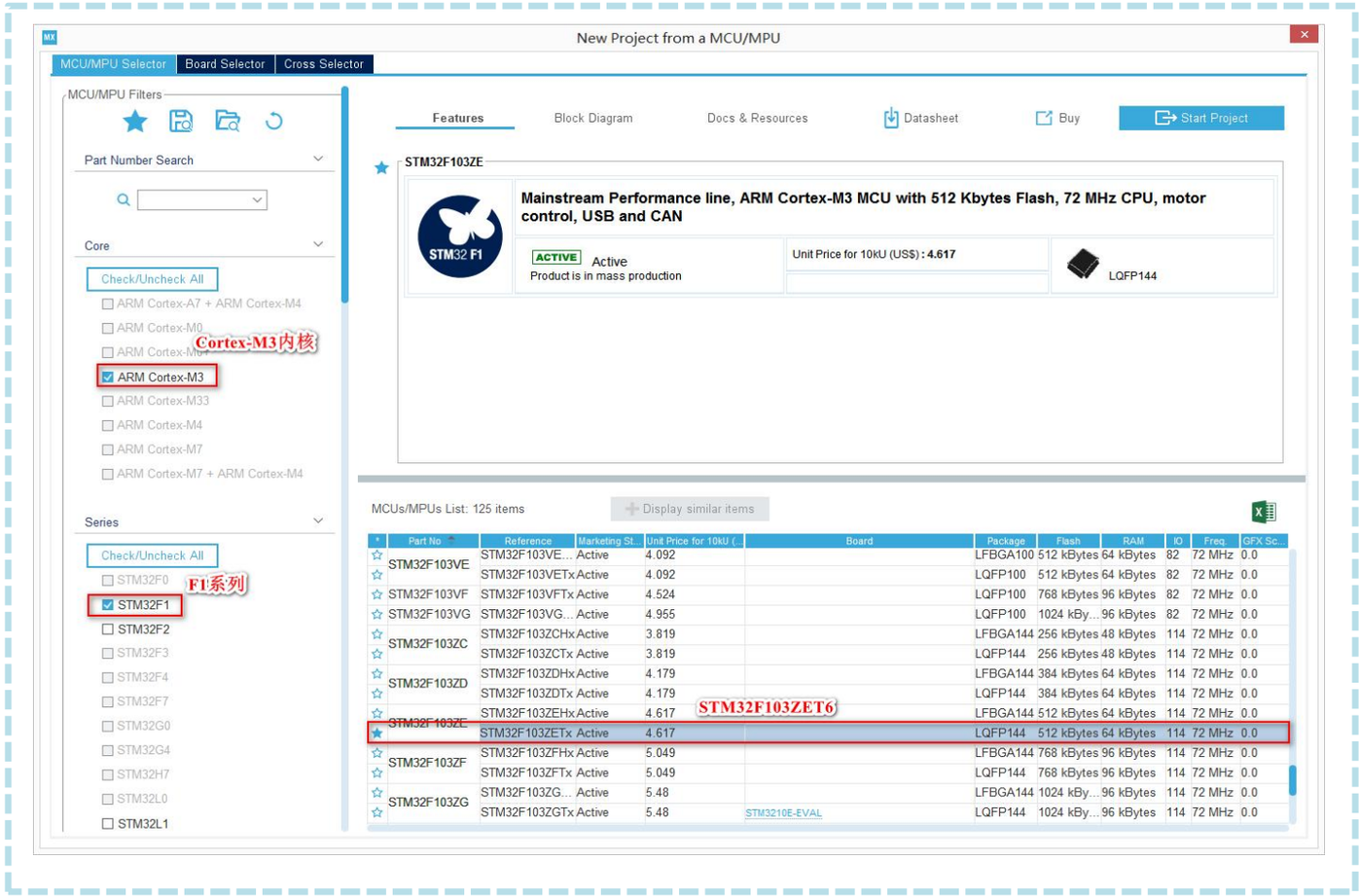
9.5.2 定时器HAL库应用实例

软件设计流程



软件设计——新建STM32CubeMX工程，选择MCU

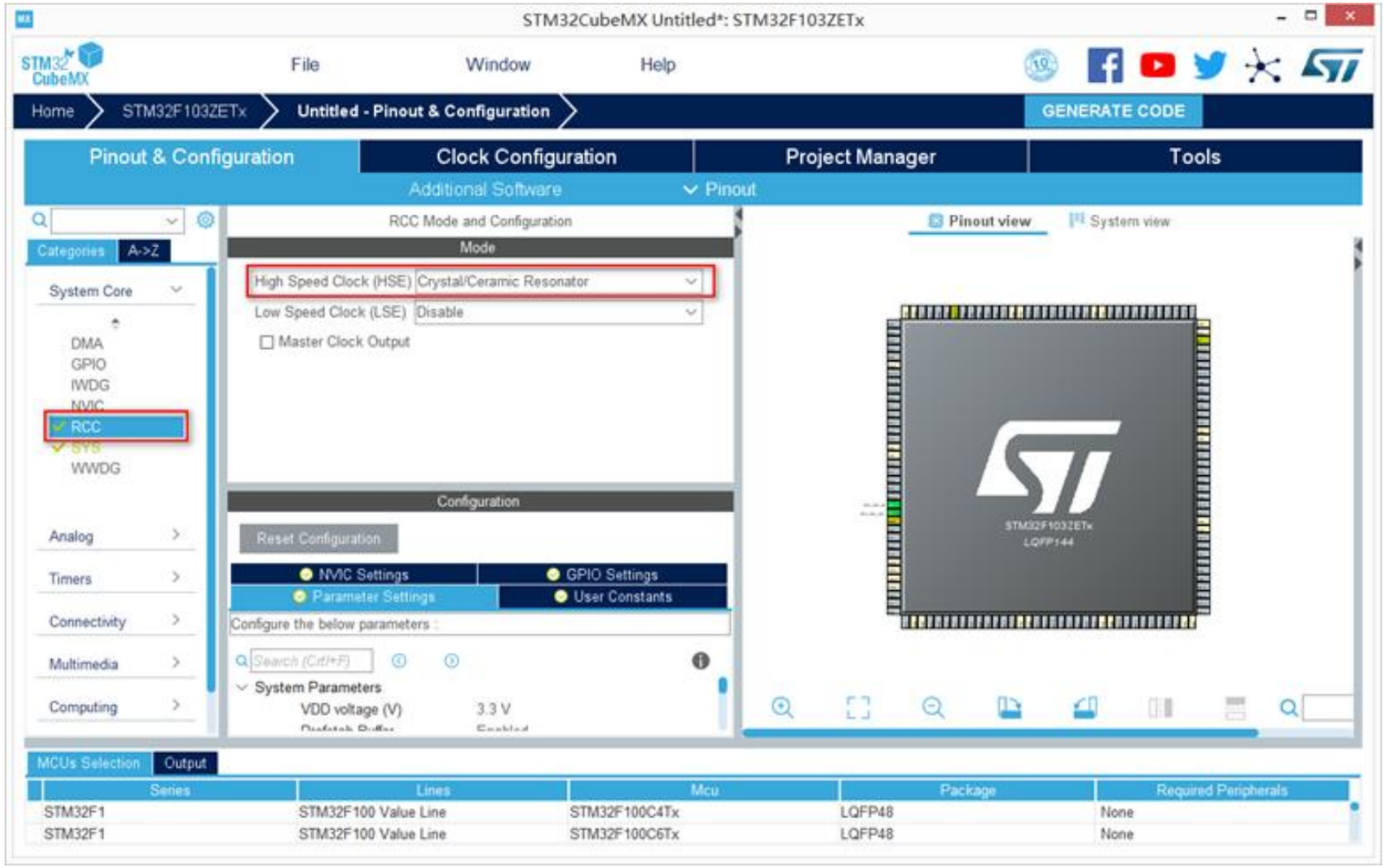
新建一个STM32CubeMX工程，选择MCU，这里采用STM32F103ZET6芯片。



软件设计——STM32CubeMX功能参数配置

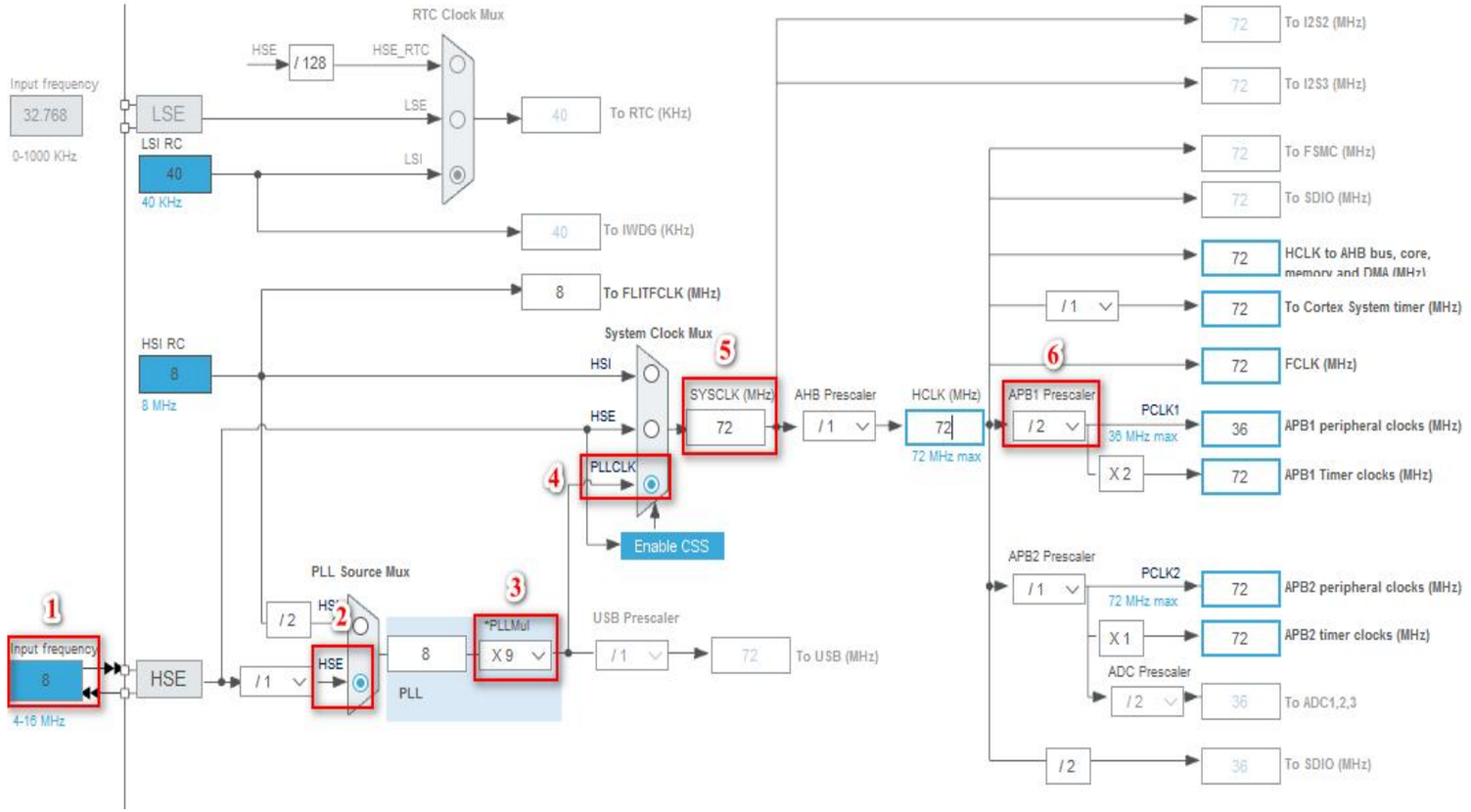
RCC配置

HSE选择“Crystal/Ceramic Resonator”(晶振/陶瓷谐振器），LSE选择“Disable”



软件设计——STM32CubeMX功能参数配置

时钟配置



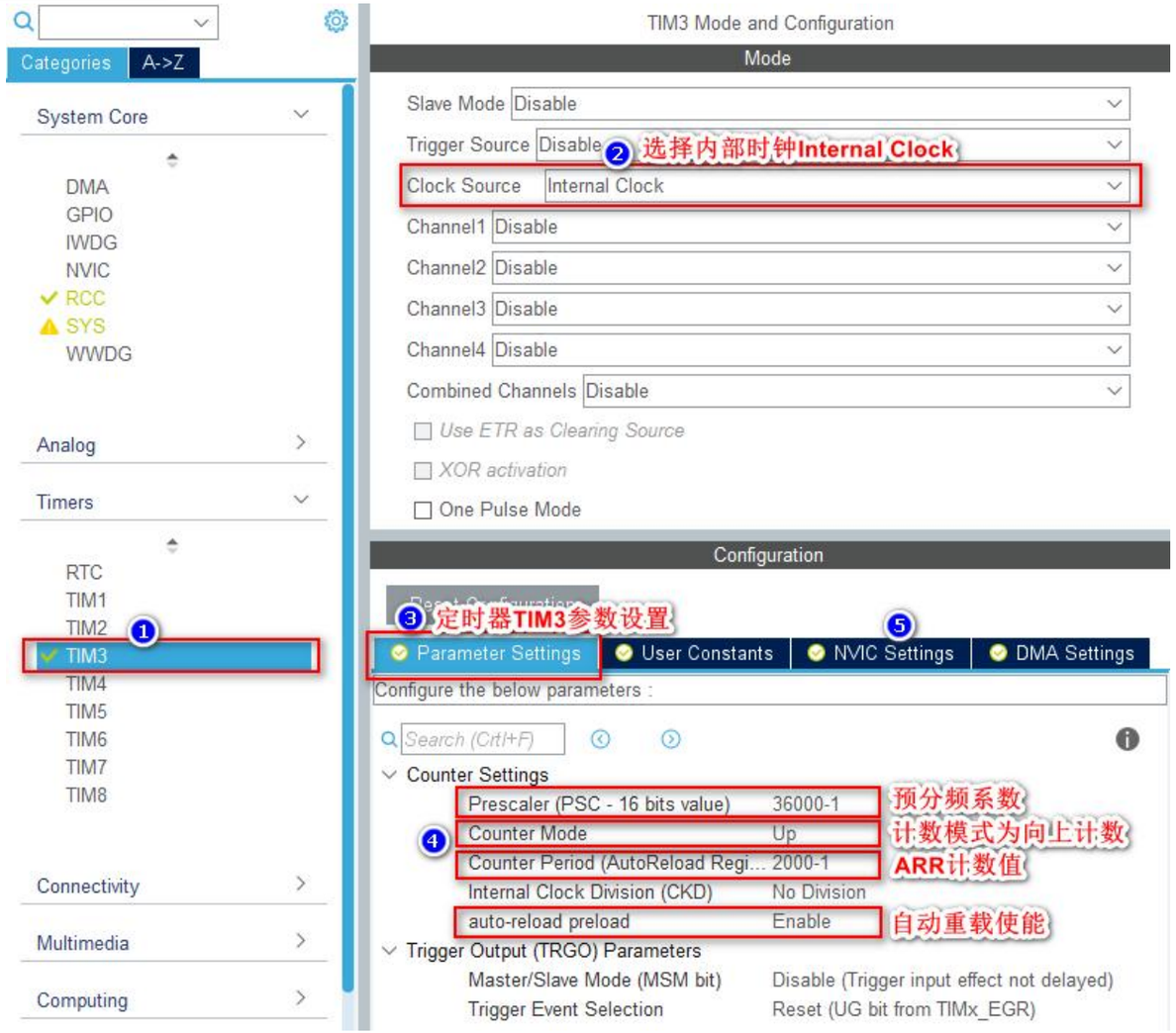
配置STM32的时钟系统，系统时钟配置为72MHz，APB2为72MHz，配置APB1为36MHz。

软件设计——STM32CubeMX功能参数配置

定时器TIM3配置

- “Clock Source”设置为“Internal Clock”(内部时钟）；
- 预分频系数“Prescaler”设置为“36000-1”；“Counter Mode”设置为“Up”；
- 计数值“Counter Period”设置为“2000-1”；
- “auto-reload preload”设置为“Enable”。

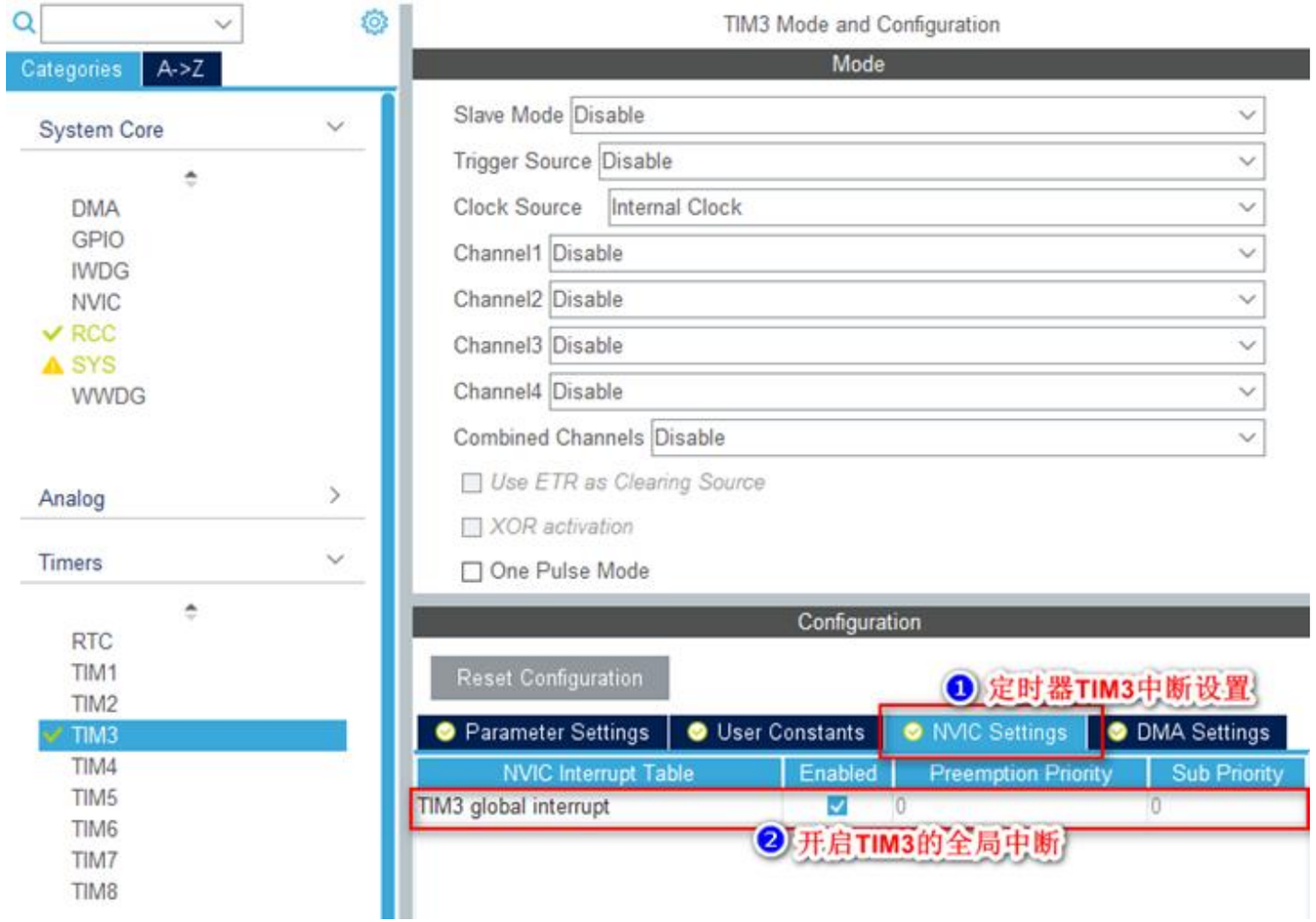
则定时器TIM3精确定时1s。



软件设计——STM32CubeMX功能参数配置

定时器TIM3配置

- 在定时器TIM3参数配置“Configuration”选项配置页的“NVIC Settings”中使能定时器TIM3的全局中断

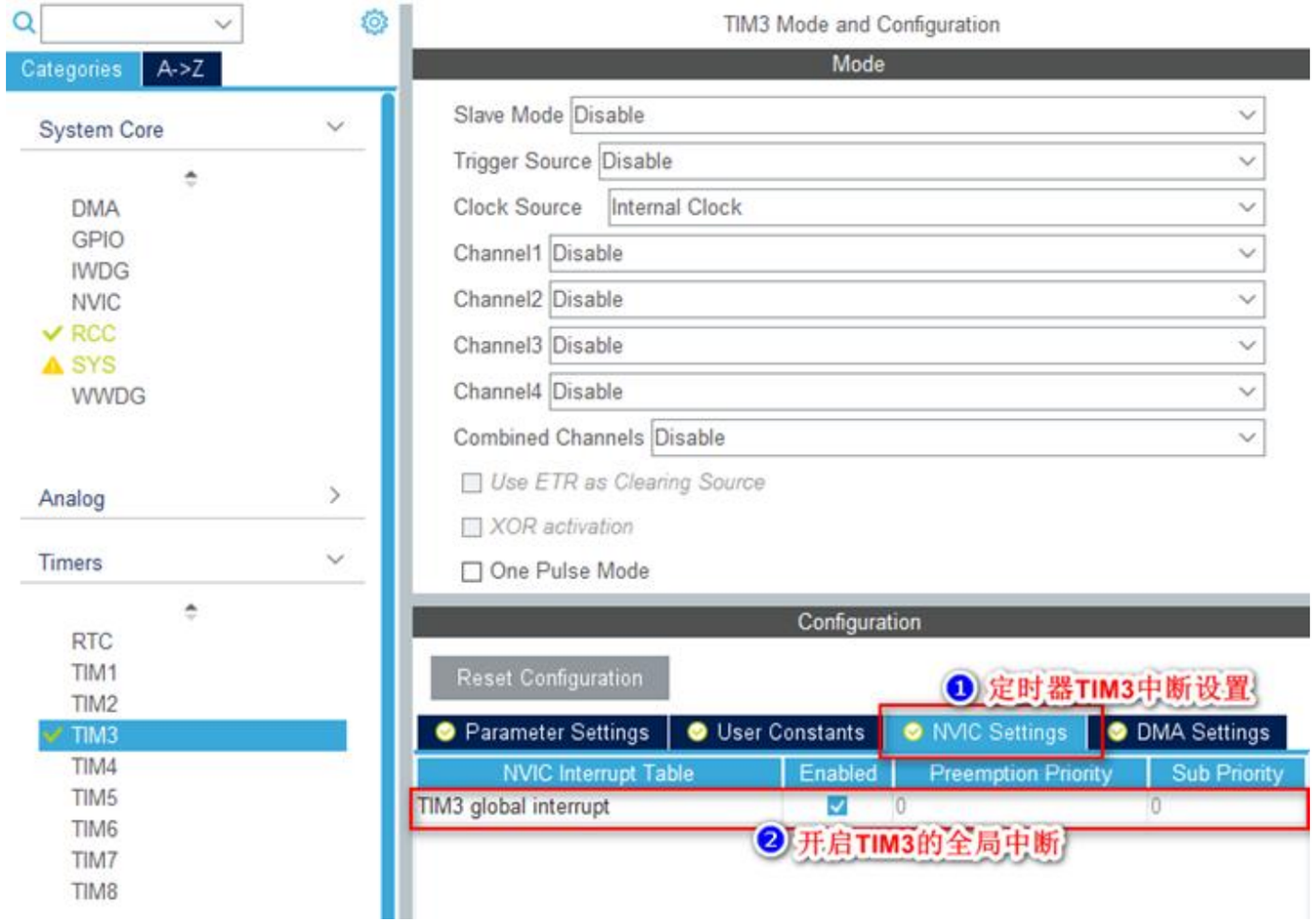


使能TIM3的全局中断

软件设计——STM32CubeMX功能参数配置

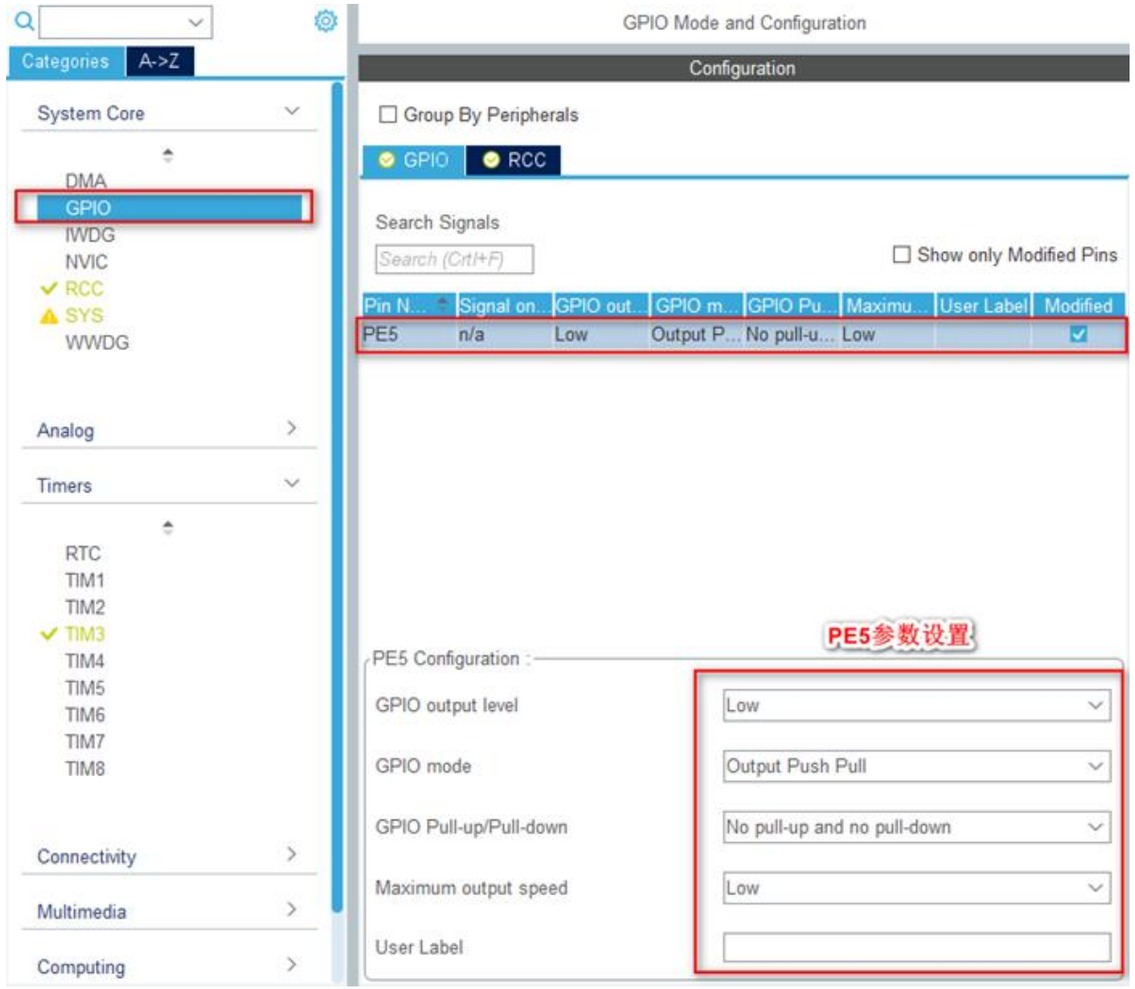
定时器TIM3配置

- 在定时器TIM3参数配置“Configuration”选项配置页的“NVIC Settings”中使能定时器TIM3的全局中断



使能TIM3的全局中断

软件设计——STM32CubeMX功能参数配置

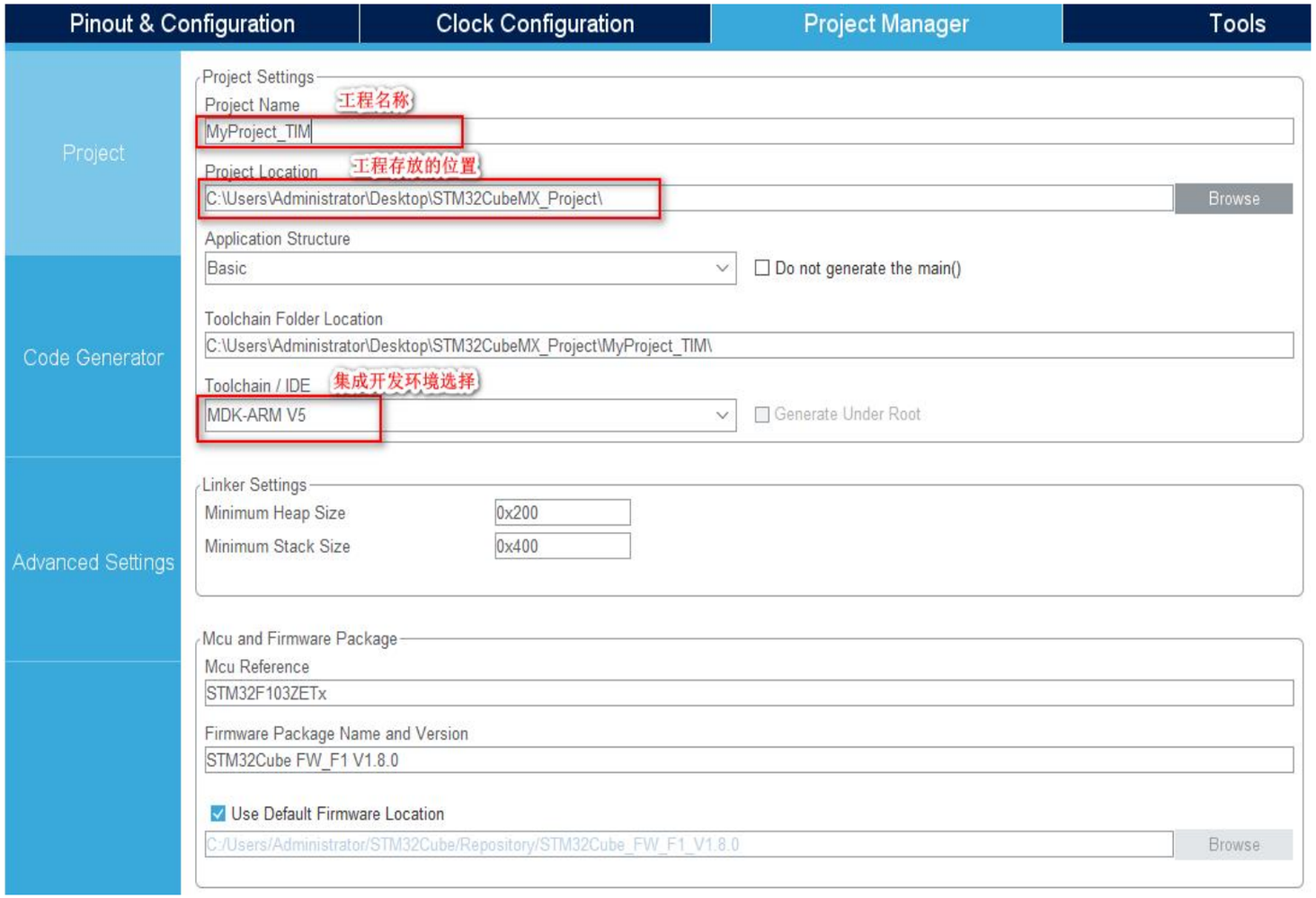


GPIO引脚设置

- 设置PE5引脚为“GPIO_Output”，用于本例的LED指示灯显示，并在“GPIO Mode and Configuration”中配置相应的参数

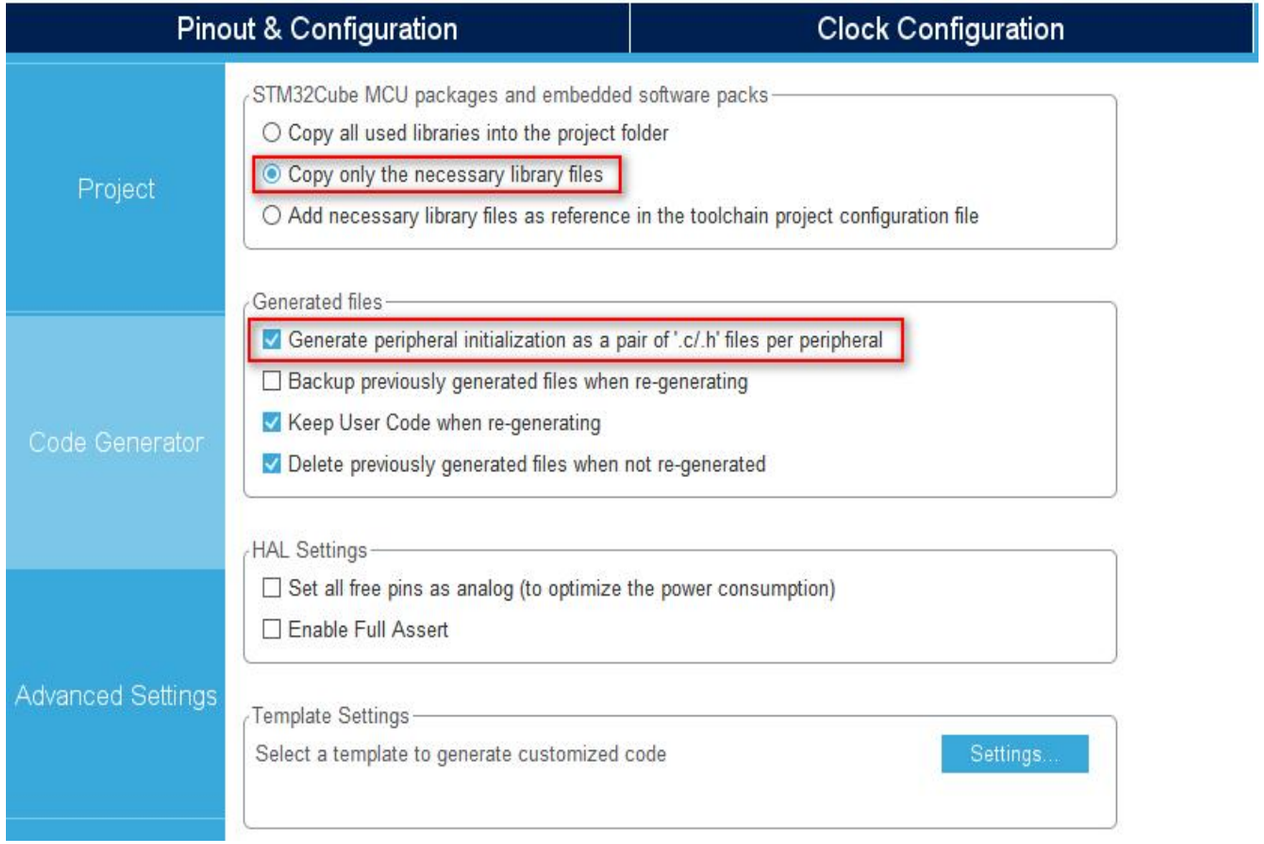
软件设计——生成工程代码

工程相关参数配置



配置工程名称、工程保存位置、选择“MDK-ARM V5”编译器等，生成工程代码

软件设计——生成工程代码



在“Code Generator”选项栏中找到“Generated files”框，勾选“Generate peripheral initialization as a pair of ’.c/.h’ files per IP”，将外设初始化的代码设置生成为独立的.c源文件和.h头文件。

软件设计——生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计——编写应用程序

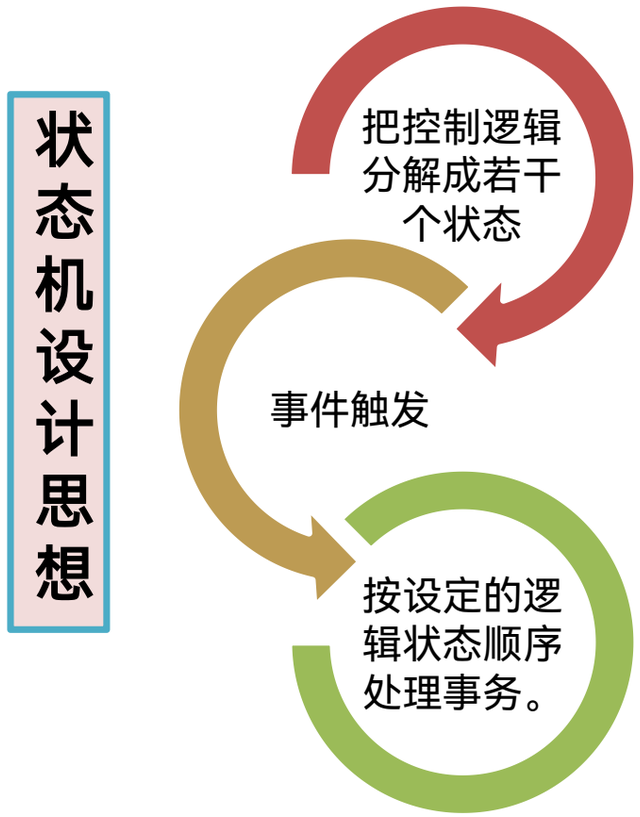
在main.c文件中的/*USER CODE BEGIN2 */和/*USER CODE END2 */之间添加开启定时器TIM3中断的程序

```
MX_TIM3_Init();
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim3);
//启动定时器TIM3定时中断
/* USER CODE END 2 */
```

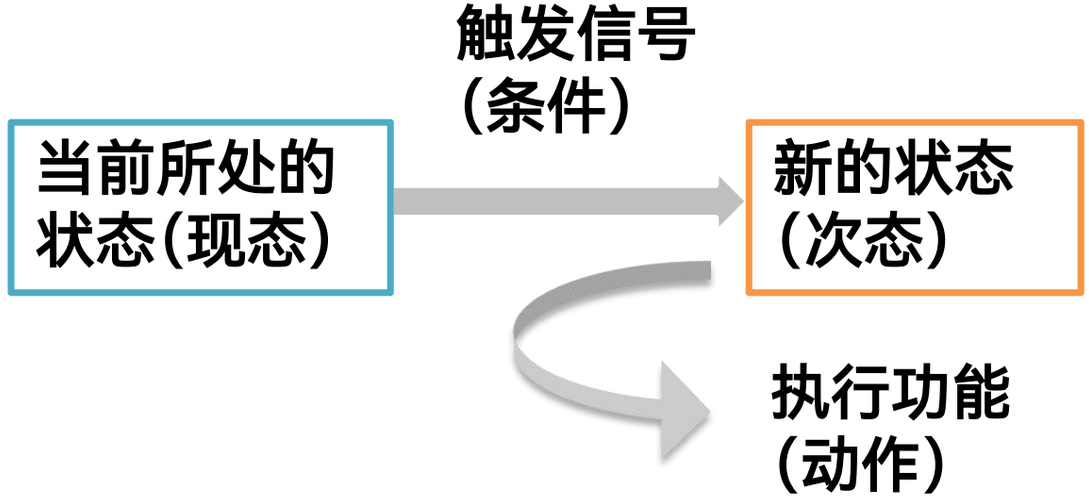
```
/* USER CODE BEGIN 4 */
void
HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef
*htim)
{
    if(htim->Instance == htim3.Instance)
    {    HAL_GPIO_TogglePin(GPIOE,GPIO_PIN_5);
    }
}
/* USER CODE END 4 */
```

在mian.c文件的/*USER CODE BEGIN4 */和/*USER CODE END4 */之间添加TIM3的中断回调函数

9.6 编程思想之状态机设计思想



嵌入式开发中，系统往往由有限个状态组成，因此状态机又称为有限状态机(Finite State Machine，FSM）。

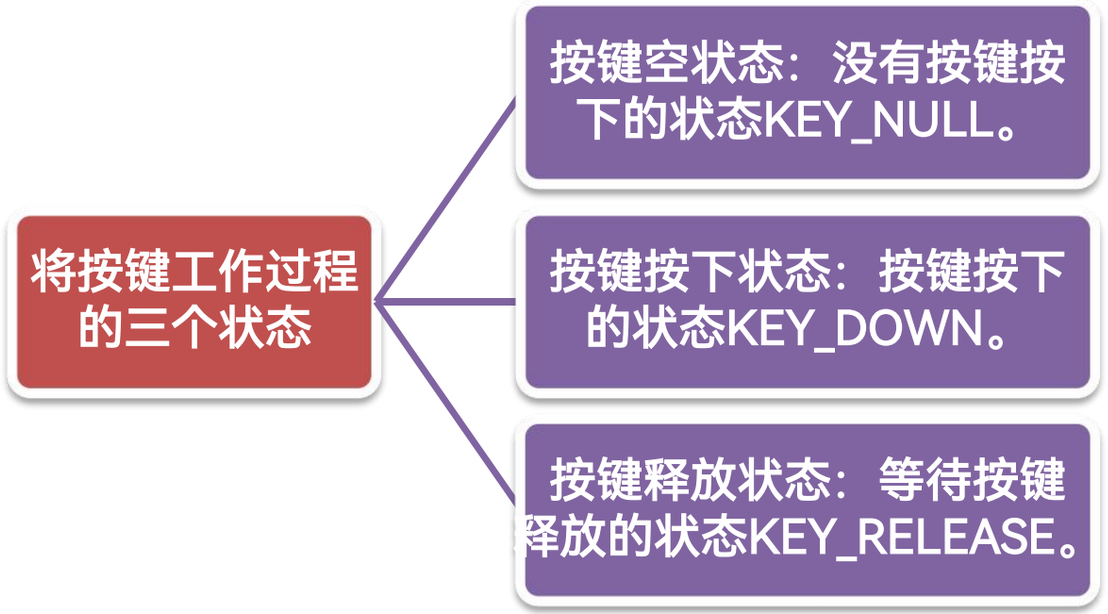


9.6 编程思想之状态机设计思想

状态	条件/事件	动作
系统在某个时间点所处的形态或状况.根据状态在系统中所处时间阶段的不同，分为现态和次态。现态即系统当前所处的状态，次态表示当某个条件满足时要转移的新状态。	用于触发状态机进行状态转换、产生相应动作(输出）。当某个条件满足时将会执行一次状态的迁移或触发一个动作。	表示条件满足后具体执行的操作。动作要素不是必需的，当条件满足后，也可以不执行任何动作，直接迁移到新状态。

9.6 编程思想之状态机设计思想

基于状态机设计思想实现按键处理程序，利用定时器每隔20ms产生一次中断，在定时中断服务程序中调用按键状态转换程序。



9.6 编程思想之状态机设计思想

按键状态转换

现态	输入	次态	输出
KEY_NULL	按键按下(按键端口电平为0)	KEY_DOWN	
	按键按下(按键端口电平为1)	KEY_NULL	
KEY_DOWN	按键输入信号为1	KEY_NULL	
	按键输入信号为0	KEY_RELEASE	产生相应的按键标志
KEY_RELEASE	按键输入信号为1	KEY_NULL	
	按键输入信号为0	KEY_RELEASE	

9.6 编程思想之状态机设计思想

定义一个枚举类型列出按键的所有状态，添加在由STM32CubeMX生成的Keil工程的主程序中。首先，定义按键状态的数据类型，采用枚举类型，用3个常量表示按键的三种状态。

```
  /* Private typedef */
  /* USER CODE BEGIN PTD */
typedef enum
{
   KEY_NULL = 0, //按键未按下
   KEY_DOWN,     //有按键按下
   KEY_RELEASE    //按键释放状态
} KEY_STATES;
   /* USER CODE END PTD */
```

9.6 编程思想之状态机设计思想

用按键枚举类型KEY_STATES定义一个变量keyState用于表示按键的状态，定义一个标志keyFlag用于表示按键是否有效。

```
/* USER CODE BEGIN PV */
KEY_STATES  keyState=KEY_NULL;
  //按键状态，初值为按键空状态
uint8_t  keyFlag=0;
  //按键值有效标志，1为有效；0为无效
/* USER CODE END PV */
```

使能定时器中断，根据所使用的定时器，进行设置并开启定时器。

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim1); //使能定时器1更新中断，启动定时器1
/* USER CODE END 2 */
```

9.6 编程思想之状态机设计思想

定义void Key_FSM(void)函数

```
 /* USER CODE BEGIN 4 */
void Key_FSM(void)       //按键状态检测函数
{
   switch(keyState)     //按键状态检测
   {    case KEY_NULL:   //如果是初始状态，即按键未被按下
        {
        //如果读到低电平，说明按键按下，则进入按键按下状态，否则保持当前状态不变
        if(HAL_GPIO_ReadPin(KEY_GPIO_Port,KEY_Pin) == GPIO_PIN_RESET)
        {          keyState=KEY_DOWN;
        }
        break;
        }
        case KEY_DOWN:   //按键按下状态
        {
        //如果读到低电平，说明按键按下，切换到按键释放状态，并设置标志位
        if(HAL_GPIO_ReadPin(KEY_GPIO_Port,KEY_Pin) == GPIO_PIN_RESET)
          {
             keyState=KEY_ RELEASE;
             keyFlag=1;//设置有效按键标志，表示按键按下后立即执行按键任务
          }
```

```
else   //如果读到高电平，则认为是干扰信号，切换到按键空状态
          {          keyState=KEY_NULL;
          }
          break;
        }
        case KEY_RELEASE:  //按键释放状态
        {    //若读到高电平，则切换到按键空状态，否则保持当前状态

if(HAL_GPIO_ReadPin(KEY_GPIO_Port,KEY_Pin)==GPIO_PIN_SET)
          {          keyState=KEY_NULL;
          }
          break;
        }
        default: break;
   }
}
        //定时器中断回调函数
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{   if(htim->Instance==TIM1)  //判断中断来源
   {
      void Key_FSM(void) ;
   }
}
/* USER CODE END 4 */
```

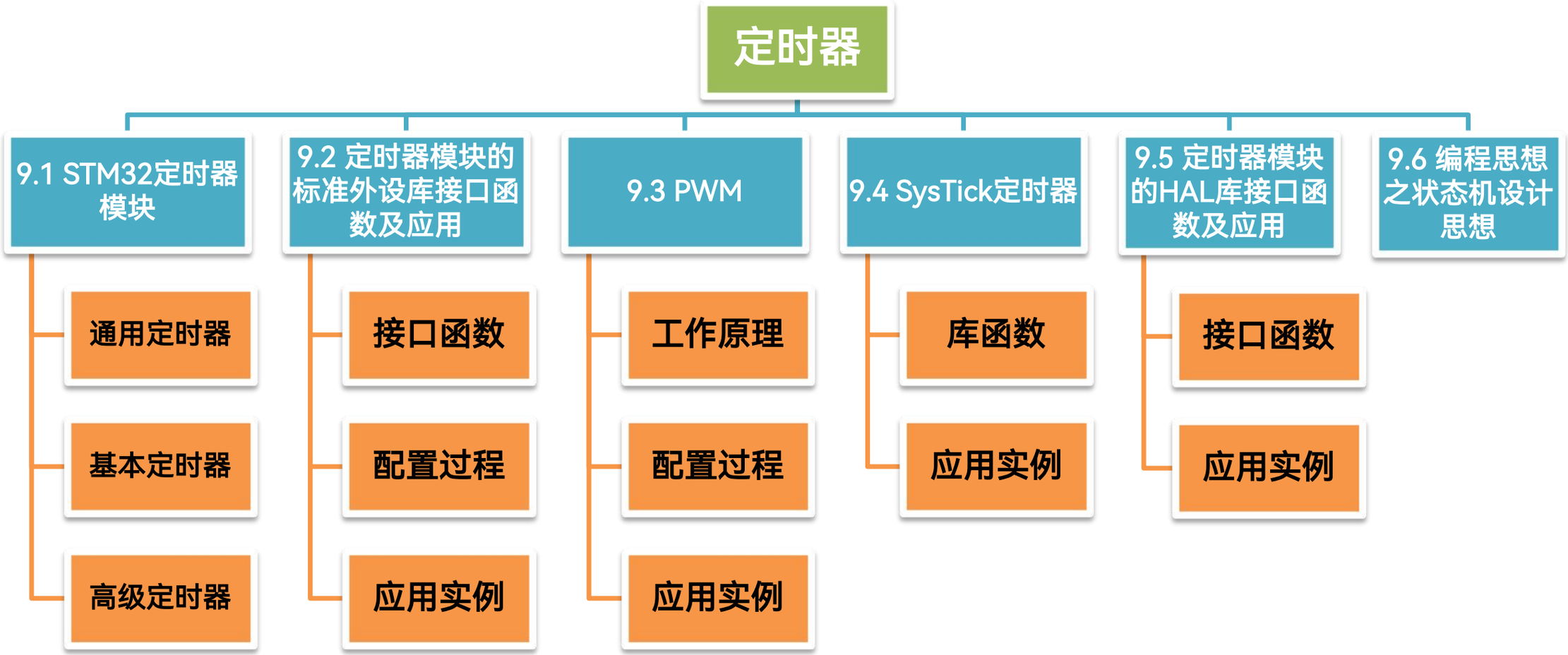
9.6 编程思想之状态机设计思想

最后编写后台检测程序，根据标志位执行指示灯状态翻转的操作。

```
    /* USER CODE BEGIN WHILE */
  while (1)
  {
      /* USER CODE END WHILE */
      /* USER CODE BEGIN 3 */
   if(keyFlag == 1）//检测按键有效标志
   {
      keyFlag=0;  //清除标志位

HAL_GPIO_TogglePin(LED1_GPIO_Port,LED1_Pin);
       //执行按键任务，指示灯状态翻转
   }
  }
      /* USER CODE END 3 */
}
```

本章小结


定时器
9.1 STM32定时器模块
通用定时器
基本定时器
高级定时器
9.2 定时器模块的标准外设库接口函数及应用
接口函数
配置过程
应用实例
9.3 PWM
工作原理
配置过程
应用实例
9.4 SysTick定时器
库函数
应用实例
9.5 定时器模块的HAL库接口函数及应用
接口函数
应用实例
9.6 编程思想之状态机设计思想



The End!

